



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

**CORSO DI LAUREA IN INFORMATICA
INGEGNERIA DEL SOFTWARE**

Anno accademico 2025/2026

Progettazione e sviluppo di un prodotto software 

BugBoard26

Autore

Nome Vittorio Emanuele Testa
Matricola N86/4823
E-mail vitt.testa@studenti.unina.it

Docenti

Nome Porfirio Tramontana
Breve Bernardo

Indice

1	Revisioni	1
2	Introduzione	2
2.1	Descrizione del prodotto	2
2.2	Obiettivi del sistema	2
2.3	Contesto d'uso	2
2.4	Glossario	3
3	Documento di Specifica dei Requisiti Software	6
3.1	Prospettiva del prodotto	6
3.2	Funzionalità del sistema	6
3.3	Caratteristiche degli utenti	6
3.4	Use Case UML	7
3.4.1	Descrizione testuale strutturata	8
3.4.2	Mock-Ups	10
3.5	Modello del Dominio del Problema	12
3.5.1	Segnalare Issue	12
3.5.2	Assegnare Bug	12
3.5.3	Creare Utenza	13
3.5.4	Effettuare Login	13
3.5.5	Consultare Riepilogo Issue	14
3.5.6	Visualizzare Bug Assegnati	15
3.5.7	Visualizzare Cronologia Bug	15
3.5.8	Modificare Bug	16
3.6	Personas	17
3.6.1	Persona 1	17
3.6.2	Persona 2	18
3.7	Requisiti Non Funzionali	19
3.8	Requisiti e Vincoli di Dominio	19
3.9	Assunzioni e dipendenze	20
4	Documento di Design del Sistema	21
4.1	Decomposizione del Sistema e Topologia dei Container	21
4.2	Analisi delle Scelte Architetture e Trade-Off	21
4.2.1	Architettura Back-End, Front-End e Livello di Persistenza	21
4.2.2	Pattern Comunicativi e Strutturali	23
4.3	Modellazione UML e Progettazione di Dettaglio	24
4.3.1	Deployment Diagram: Topologia e Hardening Infrastrutturale	24
4.3.2	Class Diagram: <code>auth-service</code>	25
4.3.3	Component Diagram: Architettura del <code>core-service</code>	26
4.3.4	Class Diagram: Sottosistemi del <code>core-service</code>	27
4.3.5	Modellazione Comportamentale e Dinamica del Back-End	31
4.3.6	Component Diagram: Moduli Front-End Angular	34
4.3.7	Schema per la Persistenza Dati e Diagramma ER	35
5	Containerizzazione e Infrastruttura di Deployment	39
5.1	Build delle Immagini: Multi-Stage e Sicurezza	39
5.2	Orchestrazione dei Servizi: Docker Compose	41
5.3	Gestione dei Volumi e Sincronizzazione dell'Avvio	42

6	Versioning e Gestione del Codice Sorgente	43
6.1	Sistema di Version Control	43
6.2	Strategia di Sviluppo e Integrazione	43
6.3	Statistiche del Repository	44
6.4	Evidenza della Cronologia Git	45
7	Code Quality e Analisi Statica	46
7.1	Strumenti di Analisi	46
7.2	Architettura dell'Analisi CI/CD	46
7.3	Risultati dell'Analisi	46
8	Attività di Testing	47
8.1	Input Space Partitioning Functionality-Based	47
8.2	Classi di equivalenza individuate	47
8.2.1	Metodo: assignBug(Long id, AssignBug request)	47
8.2.2	Metodo: updateIssue(Long id, UpdateIssue request, MultipartFile file)	48
8.2.3	Metodo: recordEvent(Long bugId, Long authorId, AuditAction action, String details)	48
8.3	Criterio di copertura: WECT (Each Choice Coverage)	49
8.4	Strategie non adottate e motivazioni	49
8.5	Metriche di Copertura Strutturale (White-Box)	50

1 Revisioni

Data	Versione	Descrizione
18/07/2026	1.0	Prima stesura del Documento di Specifica dei Requisiti Software.
20/07/2026	1.1	Aggiornata l'introduzione del documento e cambiato titolo del capitolo 3 (“Descrizione generale” → “Documento di Specifica dei Requisiti Software”).
24/07/2026	1.2	Aggiornato Documento di Specifica dei Requisiti Software e descritta prima stesura del Documento di Design del Sistema.
30/07/2026	1.3	Completamento del Documento di Design del Sistema e stesura del capitolo relativo alla Containerizzazione (Docker).
05/08/2026	1.4	Inserimento dei capitoli relativi al Versioning del codice (Git) e all'analisi della Qualità del Codice (SonarQube).
11/08/2026	1.5	Stesura del capitolo sulle Attività di Testing e preparazione alla consegna finale

2 Introduzione

La presente documentazione si struttura in tre capitoli principali: il Documento di Specifica dei Requisiti Software (SRS), il Documento di Design del Sistema e l'Attività di Testing. Il documento, nel suo complesso, descrive in modo dettagliato le funzionalità, i requisiti e le caratteristiche del sistema **BugBoard26**, nonché le procedure e le modalità con cui il prodotto è stato testato.

L'obiettivo è fornire una base chiara e condivisa tra gli sviluppatori, i committenti e gli stakeholder, utile per la progettazione, l'implementazione e la verifica del prodotto software.

2.1 Descrizione del prodotto

Il prodotto software **BugBoard26** consiste in una piattaforma per la gestione collaborativa delle *issue* nei progetti di sviluppo software. Il sistema permette ai membri di un team di tracciare e amministrare l'intero ciclo di vita delle segnalazioni, facilitando la comunicazione, l'assegnazione dei carichi di lavoro e il monitoraggio delle tempistiche. La piattaforma prevede due livelli di accesso distinti, utenti standard e amministratori, al fine di delineare chiaramente le responsabilità operative e garantire la sicurezza, la tracciabilità e la riservatezza dei dati trattati.

2.2 Obiettivi del sistema

Gli obiettivi principali del sistema sono:

- migliorare la collaborazione tra i membri del team di sviluppo;
- centralizzare la gestione delle issue e delle informazioni relative;
- fornire un'interfaccia performante e facile da usare.

2.3 Contesto d'uso

BugBoard26 è pensato per essere utilizzato in contesti di sviluppo software di piccole e medie dimensioni, garantendo accesso multiutente e sincronizzazione in tempo reale delle informazioni.

2.4 Glossario

Di seguito, sono riportati i principali termini e acronimi utilizzati nel presente Documento di Specifica dei Requisiti Software. Il glossario ha lo scopo di fornire definizioni chiare e univoche per facilitare la comprensione del testo e garantire coerenza terminologica. In caso di dubbi relativi a uno specifico termine, si faccia riferimento alla seguente tabella, che mi auguro essere esaustiva ed esplicativa.

Ruoli e Attori	
Admin	Ruolo con privilegi di gestione e configurazione del sistema, comunemente noto come superuser.
Utente	Attore standard autenticato all'interno del sistema, con privilegi operativi limitati.

Termini Tecnici e di Dominio	
Bug	Difetto o errore logico nel software che causa un comportamento anomalo o non previsto.
Cronologia	Registro storico che tiene traccia degli eventi e delle modifiche apportate a un'entità nel corso del tempo.
Documentation	Tipologia di segnalazione utilizzata per richiedere la stesura, l'aggiornamento o la revisione di manualistica e documentazione.
Feature	Tipologia di segnalazione utilizzata per proporre lo sviluppo di una nuova funzionalità o caratteristica software.
Issue	Termine generale che indica qualsiasi ticket o segnalazione tracciabile all'interno di un progetto.
Notifica	Messaggio di avviso informativo generato dal sistema e recapitato a un utente a seguito di un evento rilevante.
Progetto	Ambiente logico di riferimento all'interno del quale un team di sviluppo organizza il proprio lavoro.
Question	Tipologia di segnalazione utilizzata per porre domande o richiedere chiarimenti su aspetti del software.
Stateless	Paradigma architetturale in cui il server non conserva in memoria lo stato della sessione del client tra richieste HTTP distinte.

Tecnologie e Strumenti	
Angular	Framework open-source basato su TypeScript, utilizzato per lo sviluppo di applicazioni web lato client.
API	Application Programming Interface. Insieme di definizioni e protocolli mediante i quali diversi software comunicano tra loro.
Back-end	Livello architetturale di un'applicazione responsabile della logica di business, dell'elaborazione e della persistenza dei dati.
Cloud Computing	Modello di erogazione di risorse informatiche (storage, calcolo) sotto forma di servizi distribuiti via Internet.
Database	Sistema organizzato per la persistenza, l'archiviazione e la gestione strutturata dei dati.
Docker	Piattaforma open-source di containerizzazione che permette di pacchettizzare un'applicazione e le sue dipendenze in un'unità standardizzata.
Front-end	Livello architetturale di un'applicazione con cui l'utente interagisce direttamente (interfaccia grafica).
GitHub	Piattaforma di hosting per lo sviluppo di software basata sul sistema di controllo di versione distribuito Git.
Hibernate / JPA	Framework per il mapping oggetto-relazionale (ORM) utilizzato in ambiente Java per gestire l'interazione con database relazionali.
Linguaggio OO	Linguaggio di programmazione basato sul paradigma Object-Oriented, che organizza il codice in classi e oggetti interagenti.
PostgreSQL	Sistema di gestione di database relazionali ad oggetti (ORDBMS), open-source e basato sullo standard SQL.
REST	Representational State Transfer. Stile architetturale basato sul protocollo HTTP per la creazione di servizi web scalabili.
SPA	Single Page Application. Applicazione web che carica un singolo documento HTML e aggiorna dinamicamente i contenuti senza ricaricare l'intera pagina.
Spring Boot	Framework Java open-source concepito per semplificare la creazione di applicazioni web e microservizi stand-alone.
Visual Paradigm	Strumento software di tipo CASE (Computer-Aided Software Engineering) utilizzato per la modellazione visiva UML.

Sicurezza informatica	
Autenticazione	Processo di verifica informatica volto a confermare e garantire l'identità di un utente o di un sistema.
Autorizzazione	Processo successivo all'autenticazione che verifica e determina i permessi di accesso a specifiche risorse di sistema.
BCrypt	Algoritmo e funzione di hash crittografico progettata appositamente per la memorizzazione sicura delle password.
CSRF	Cross-Site Request Forgery. Attacco informatico che costringe un utente a eseguire azioni non volute su un'applicazione in cui è autenticato.
JWT	JSON Web Token. Standard aperto per trasmettere in modo compatto e sicuro informazioni tra le parti sotto forma di oggetto JSON.
SQL Injection	Tecnica di attacco che sfrutta vulnerabilità del codice per iniettare comandi SQL malevoli ed eseguire interrogazioni illecite.
XSS	Cross-Site Scripting. Vulnerabilità di sicurezza che permette di iniettare script eseguiti lato client nelle pagine web.

Modellazione e Progettazione	
Mock-up	Modello visivo o bozza a bassa/media fedeltà dell'interfaccia utente, utilizzato per illustrare la struttura dell'applicazione.
Persona(s)	Archetipo fittizio costruito su dati reali per rappresentare una specifica tipologia di utente target del software.
UML	Unified Modeling Language. Linguaggio di modellazione standardizzato per rappresentare visivamente il comportamento del software.
Use Case	Descrizione delle interazioni tra uno o più attori esterni e il sistema, finalizzate al raggiungimento di un obiettivo specifico.

Standard e Qualità	
GDPR	General Data Protection Regulation. Regolamento europeo relativo alla protezione dei dati personali.
ISO/IEC 25010	Standard internazionale che definisce i modelli, le caratteristiche e le metriche per valutare la qualità dei prodotti software.
xUnit	Termine generico che identifica una famiglia di framework utilizzati per la scrittura e l'esecuzione di test di unità.

3 Documento di Specifica dei Requisiti Software

3.1 Prospettiva del prodotto

BugBoard26 si colloca nel contesto degli strumenti di supporto allo sviluppo software, fornendo una piattaforma unificata per la gestione delle *issue*. Il sistema è concepito come un'applicazione indipendente, accessibile tramite interfaccia *web-based*, ma potenzialmente integrabile con altri strumenti di sviluppo.

Il prodotto adotta un'architettura client-server, in cui l'interfaccia utente comunica con un server centralizzato per la gestione dei dati relativi a utenti, issue, commenti e cronologie. Tutti i dati vengono conservati in un database relazionale per garantire consistenza e tracciabilità.

3.2 Funzionalità del sistema

Il sistema offrirà le seguenti funzionalità principali:

- autenticazione sicura tramite email e password;
- gestione delle utenze da parte degli amministratori, con possibile aggiunta di scadenze opzionali per la risoluzione dei bug;
- creazione delle issue con titolo, descrizione, tipologia, priorità e allegati;
- vista riepilogativa con possibilità di filtrare, ordinare e ricercare le issue secondo criteri multipli;
- assegnazione dei bug ai membri del team da parte degli amministratori, con relativo sistema di notifiche e area di visualizzazione personale;
- modifica dei bug nel rispetto del proprio livello di accesso;
- tracciamento della cronologia delle modifiche per i singoli bug.

Queste funzionalità sono formalizzate visivamente nello [Use Case UML](#) alla pagina seguente.

3.3 Caratteristiche degli utenti

Gli utenti del sistema sono profilati secondo un modello di controllo degli accessi basato sui ruoli, noto come Role-Based Access Control (RBAC), distinguendo due categorie gerarchiche:

Utente (Membro del team): rappresenta l'attore base autenticato. Possiede i diritti di lettura globale che gli consentono di consultare l'elenco di tutte le segnalazioni, filtrarle, ordinarle ed esaminarne il relativo storico delle modifiche. In scrittura è abilitato ad aprire nuove segnalazioni quali bug, feature, documentation e question. Può modificare lo stato e i dettagli esclusivamente dei bug di cui risulta formalmente assegnatario.

Amministratore (Admin): rappresenta un'utenza con privilegi di gestione avanzati che estende le capacità dell'Utente base. Detiene la responsabilità esclusiva di registrare nuove utenze, assegnare i bug ai vari membri del team per la risoluzione e definire le relative scadenze. Gode di diritti di modifica completi e incondizionati su qualsiasi issue o bug presente a sistema.

3.4 Use Case UML

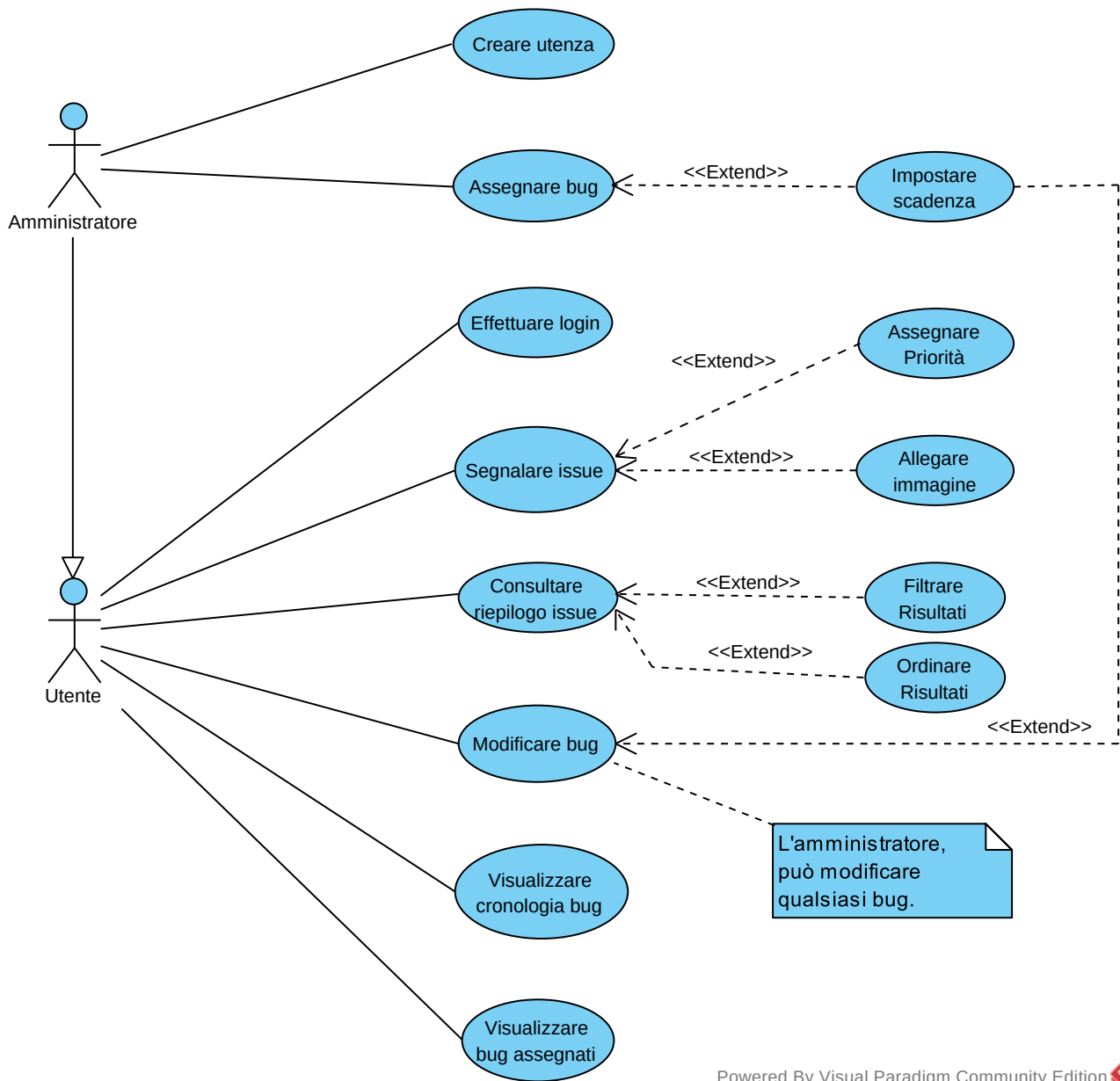


Figura 1: Use Case UML

3.4.1 Descrizione testuale strutturata

USE CASE #1	Segnalare issue		
Goal in context	Segnalare agli altri membri del team una issue su un progetto		
Preconditions	L'utente deve aver effettuato il login		
Success End Conditions	Il sistema tiene traccia della segnalazione della issue, che viene creata con stato iniziale "TODO"		
Failure End Conditions	La issue non viene aggiunta		
Primary Actor	Utente		
Trigger	L'Utente clicca l'apposito bottone per segnalare issue		
Main Scenario	Step n.	Utente	System
	1	Clicca <i>Segnala issue</i>	
	2		Mostra M02
	3	Scriva <i>Titolo</i> , <i>Descrizione</i> , <i>Progetto</i> e <i>Tipo</i>	
	4	Clicca <i>Ok</i>	
	5		Mostra M03
	6		Mostra M01
Extension #1 (clicca <i>Canc</i> e poi <i>Annulla</i>)	4a	Clicca <i>Canc</i>	
	5a		Mostra M04
	6a	Clicca <i>Annulla</i>	
	7a		Ritorna allo step 2 del main scenario
Extension #2 (clicca <i>Canc</i> e poi <i>Procedi</i>)	4b	Clicca <i>Canc</i>	
	5b		Mostra M04
	6b	Clicca <i>Procedi</i>	
	7b		Mostra M01
Extension #3 (specifica altre info)	3d	Inserisce anche priorità e allega immagine	
	4d	Clicca <i>Ok</i>	
	5d		Ritorna allo step 5 del main scenario
Extension #4 (Immagine non supportata)	4e		Mostra M05
	5e	Clicca <i>Ok</i>	
	6e		Ritorna allo step 3 del main scenario
Extension #5 (Errore generico)	4f		Mostra M06
	5f	Clicca <i>Ok</i>	
	6f		Ritorna allo step 3 del main scenario

USE CASE #2	Assegnare bug		
Goal in context	Assegnare un bug a un membro del team per la risoluzione e, opzionalmente, fissare una scadenza		
Preconditions	L'Amministratore deve aver effettuato il login. Deve esistere almeno un bug nel sistema.		
Success End Conditions	Il bug è assegnato all'utente scelto, il quale riceve una notifica. L'eventuale scadenza viene registrata.		
Failure End Conditions	L'assegnazione non viene completata. Il bug rimane nel suo stato precedente.		
Primary Actor	Amministratore		
Trigger	L'amministratore clicca sul bottone per assegnare uno specifico bug		
Main Scenario	Step n.	Admin	System
	1	Clicca <i>Assegna Bug</i>	
	2		Mostra M07
	3	Seleziona <i>Progetto, Bug</i> ed <i>Utente</i> dalla lista	
	4	Clicca <i>Ok</i>	
	5		Mostra M08 ed invia notifica all' <i>Utente</i>
	6		Mostra M01
Extension #1 (<i>Admin</i> sceglie di impostare scadenza)	3a	Inserisce data in <i>Scadenza</i>	
	4a		Ritorna allo step 5 del main scenario
Extension #2 (clicca <i>Canc</i> e poi <i>Procedi</i>)	4b	Clicca <i>Canc</i>	
	5b		Mostra M09
	6b	Clicca <i>Procedi</i>	
	7b		Ritorna allo step 6 del main scenario
Extension #3 (clicca <i>Canc</i> e poi <i>Annulla</i>)	4d	Clicca <i>Canc</i>	
	5d		Mostra M09
	6d	Clicca <i>Annulla</i>	
	7d		Ritorna allo step 2 del main scenario

3.4.2 Mock-Ups

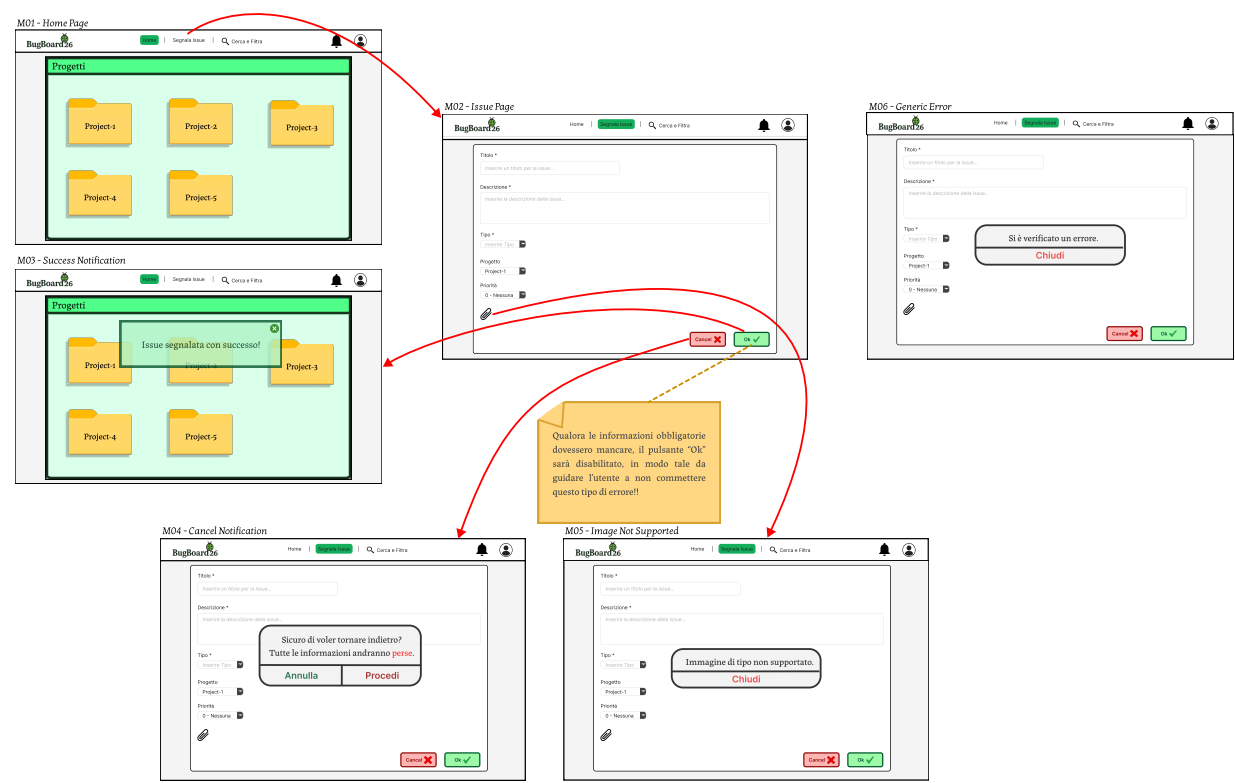


Figura 2: M01-M06

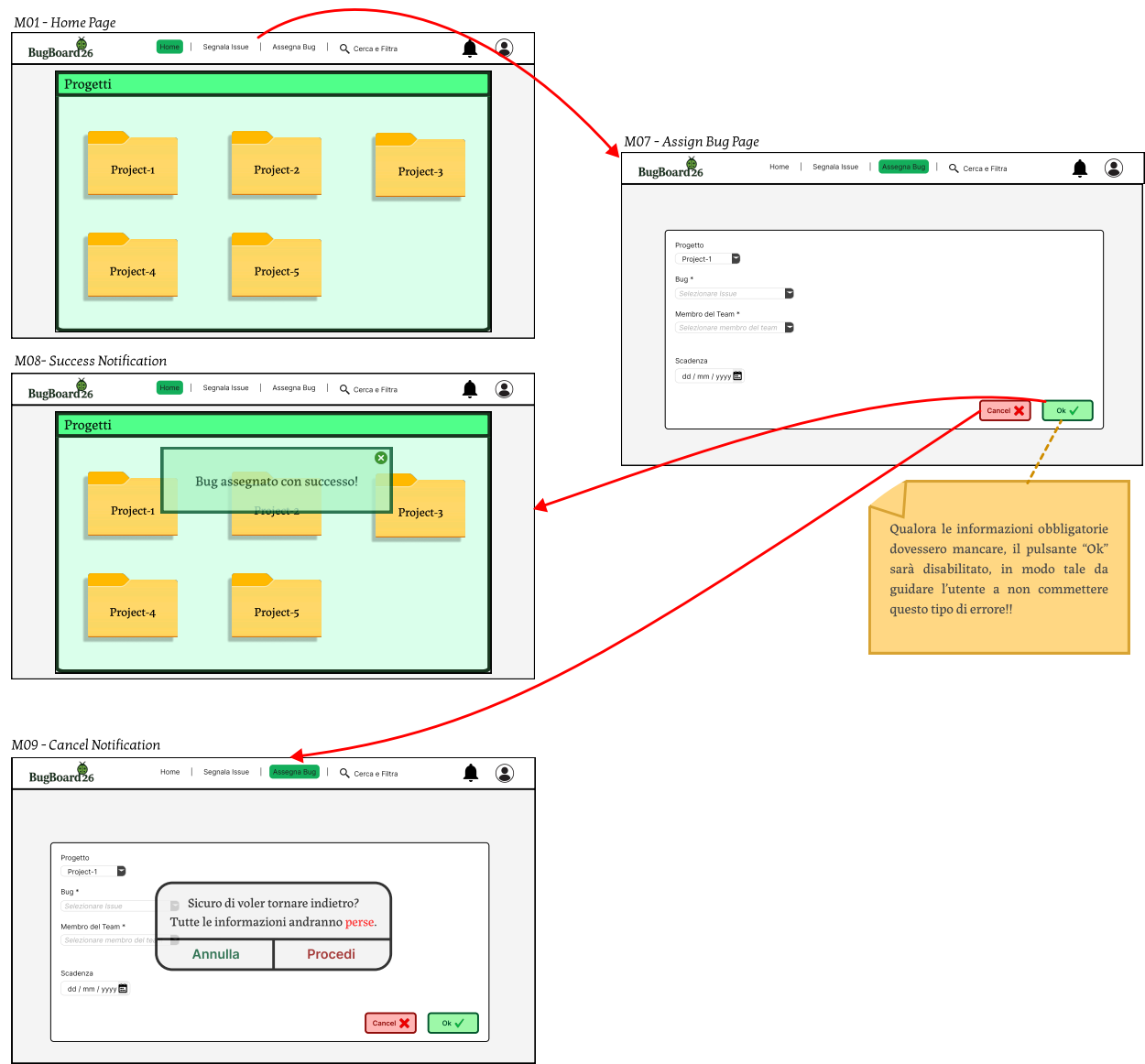


Figura 3: M01, M07-M09

3.5 Modello del Dominio del Problema

Al fine di formalizzare le entità del dominio e le loro relazioni concettuali, viene introdotto il *Modello del Dominio del Problema*.

A differenza dei diagrammi delle classi di progettazione, questo modello astrae totalmente dai dettagli tecnologici e implementativi. Si concentra unicamente sui concetti del mondo reale e sui vincoli di business.

Per gestire la complessità e garantire la massima chiarezza espositiva, la modellazione del dominio verrà presentata in maniera modulare e strutturata per Use Case: per ciascun caso d'uso verrà mostrato e descritto il relativo diagramma concettuale delle entità partecipanti.

3.5.1 Segnalare Issue

In questa prima vista concettuale vengono modellate le entità dal punto di vista dell'attore *Utente*.

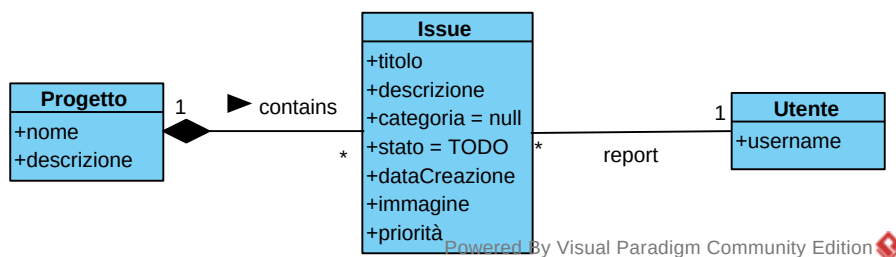


Figura 4: Segnalare Issue

3.5.2 Assegnare Bug

In questa vista vengono modellate le entità necessarie all'Amministratore per delegare la risoluzione di un Bug a un *Utente*, registrando l'eventuale *scadenza* e originando la relativa *Notifica*.

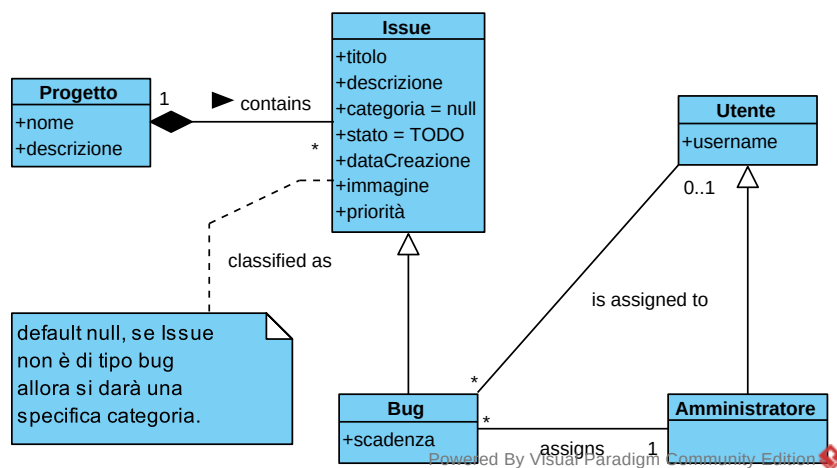


Figura 5: Assegnare Bug

3.5.3 Creare Utenza

Questa vista concettuale modella l'istanziamento di un nuovo account all'interno della piattaforma, definendo le credenziali identificative e il livello di privilegio assegnato alla specifica entità.

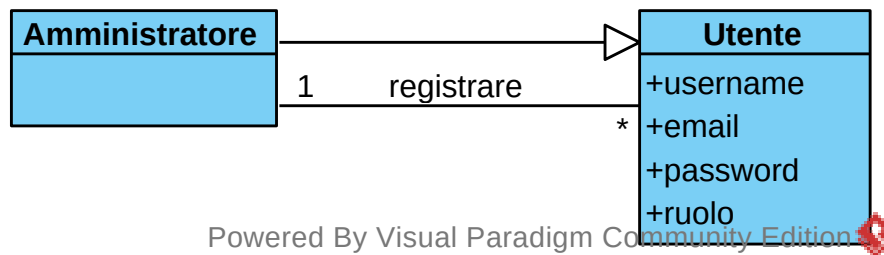


Figura 6: Creare Utenza

3.5.4 Effettuare Login

L'autenticazione all'interno del sistema si traduce, a livello concettuale, nella creazione temporanea di una **SessioneAutenticata**. Tale entità certifica il riconoscimento e abilita l'accesso operativo. Per gestire il ciclo di vita della validità dell'accesso, l'entità prevede l'attributo booleano **scaduta** con valore di default impostato a *false*.

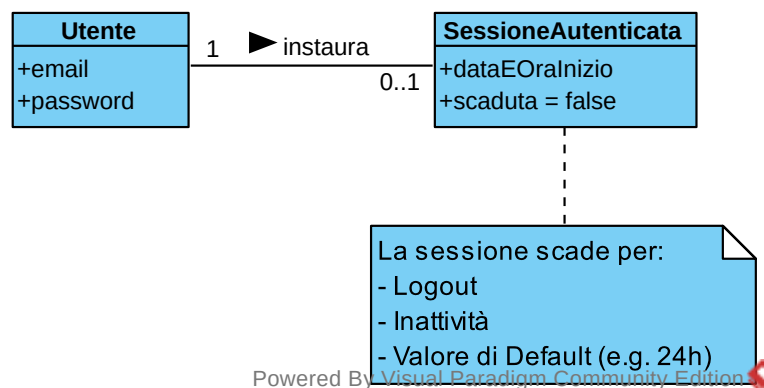


Figura 7: Effettuare Login

3.5.5 Consultare Riepilogo Issue

L'interazione di interrogazione e consultazione dell'elenco globale delle segnalazioni è mediata dalla classe associativa **Ricerca**, che permette di non inquinare le entità principali del dominio. Tale classe incapsula parametri di filtraggio opzionali. L'attributo **testoLibero** garantisce la ricerca per titolo o descrizione, mentre la proprietà **ordinamento** definisce la sequenza di visualizzazione dei risultati, applicando come default l'ordinamento storico crescente basato sulla data di creazione (**CREATED_ASC**).

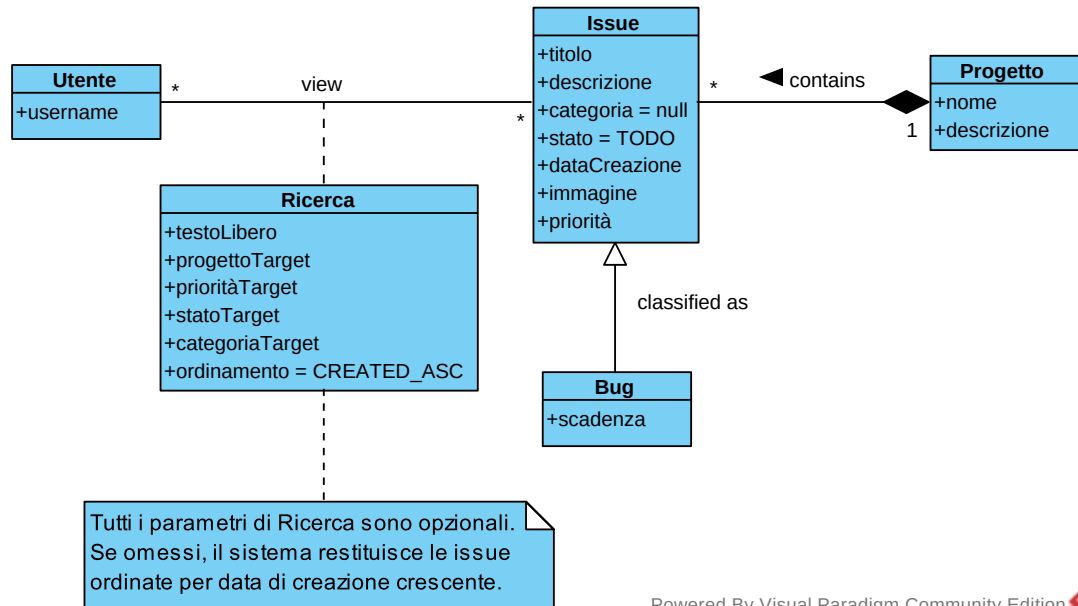


Figura 8: Consultare Riepilogo Issue

3.5.6 Visualizzare Bug Assegnati

La vista evidenzia la prospettiva personale del membro del team, mostrando la relazione con i Bug di cui risulta formalmente responsabile. La classe associativa **Notifica** caratterizza il momento dell'assegnazione, incapsulando il messaggio di avviso, il timestamp di generazione (**dataEOraInvio**) e lo stato di lettura (**letta**), strumenti fondamentali per monitorare i task aperti e le relative scadenze.

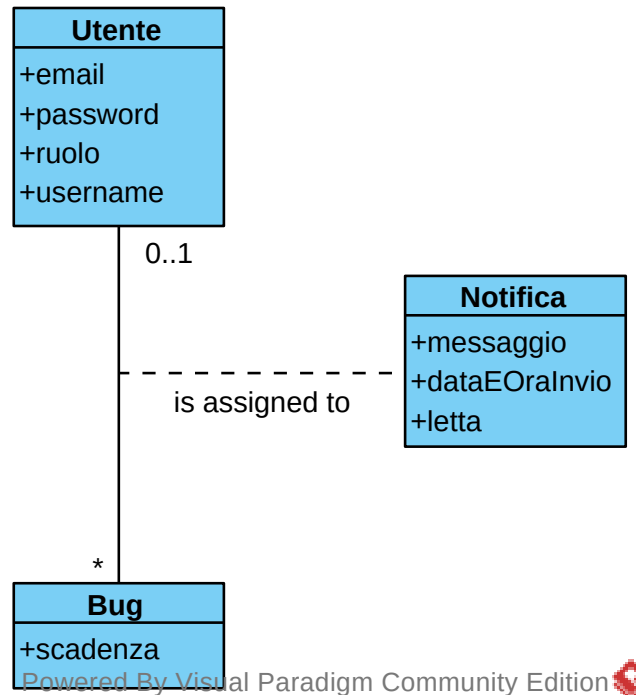


Figura 9: Visualizzare Bug Assegnati

3.5.7 Visualizzare Cronologia Bug

In questa vista viene modellata l'interazione in sola lettura per la consultazione del tracciamento storico delle modifiche subite dai bug.

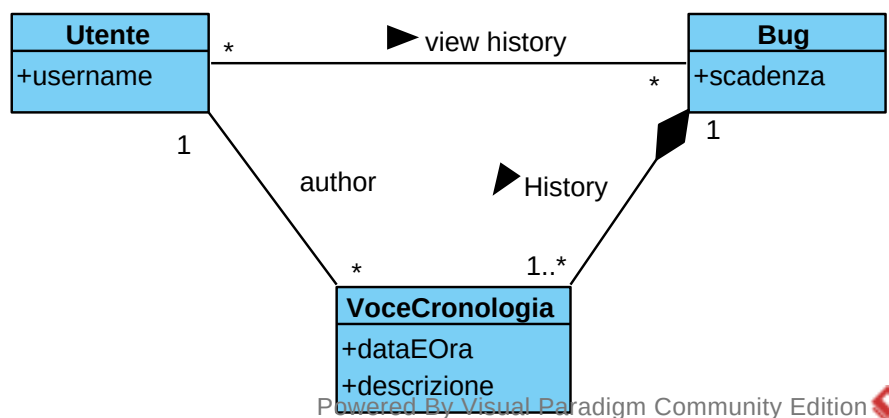


Figura 10: Visualizzare Cronologia Bug

3.5.8 Modificare Bug

Il modello traduce strutturalmente le regole di autorizzazione RBAC in relazioni di molteplicità differenti per la modifica dei dati di un bug. La relazione per l'Utente base impone una molteplicità $0..1 \leftrightarrow *$, assicurando che l'azione di modifica sia consentita esclusivamente sull'eventuale singola identità designata come assegnataria. Di contro, la relazione per l'Amministratore adotta una cardinalità multi-a-molti ($* \leftrightarrow *$), garantendo diritti di alterazione su qualunque istanza presente nel sistema. Per massimizzare l'essenzialità visiva, la superclasse **Issue** è stata intenzionalmente soppressa dal diagramma, mantenendo il focus puro sugli attori e sul Bug.

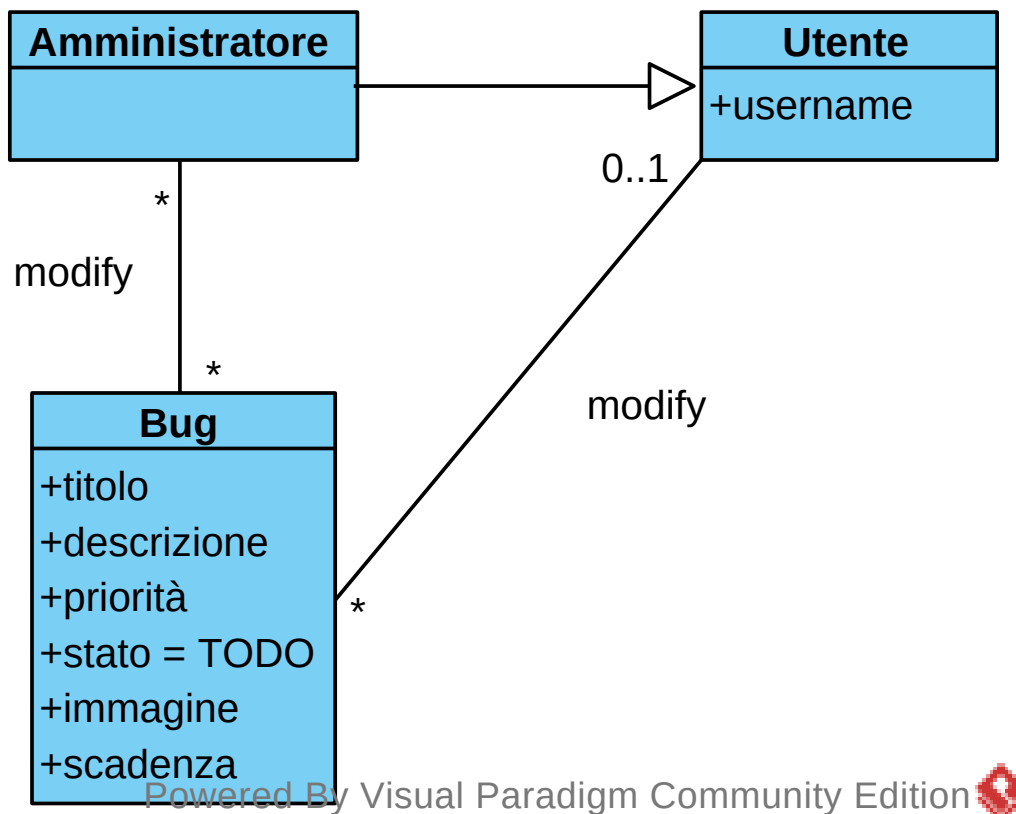


Figura 11: Modificare Bug

3.6 Personas

Al fine di identificare più chiaramente il target utenti di cui si è parlato poc'anzi, si è lavorato con lo strumento dei *Personas*. Nelle pagine seguenti sono riportati due esempi, uno per ogni tipologia di utente.

3.6.1 Persona 1



*"Efficienza è la strategia,
innovazione è lo strumento."*

Roberto Esposito

BIO

Con un approccio lungimirante e determinato, Roberto Esposito lavora come Project Manager presso la KoopaTech, in ambito IT. Laureato all'Università degli Studi di Napoli Federico II, ha intrapreso un percorso professionale che lo ha portato a gestire team e progetti in contesti dinamici e sfidanti. Ora è un leader competente e attento ai dettagli, alla ricerca di nuove tecnologie per rendere più competitiva la sua azienda.

Fuori dall'ufficio, è una persona fortemente legata alla famiglia: Roberto, infatti, è felicemente sposato ed è padre di due gemelle, con cui condivide la passione per il cinema.

About

- 48
- Napoli, Italia
- Project Manager
- Federico II, Napoli
- Maschio
- Sposato

Lungimirante Smargiasso Competitivo

FAVORITE BRANDS



GOALS

- Gestire diversi progetti in contemporanea in modo chiaro.
- Mantenere il controllo sugli account e sui permessi utente.
- Migliorare le performance aziendali.

INTERESTS

- Sport: gli piace organizzare partite di calcio con i suoi vecchi compagni.
- Cinema: trova affascinanti i film dell'orrore e i racconti d'autore.
- Famiglia: ritiene importante curare i rapporti all'interno della sua famiglia.

NEEDS

- Avere una visione complessiva dello stato dei progetti e delle issue aperte.
- Permettere, in maniera sicura, di mostrare i progetti agli stakeholder.
- Monitorare le performance aziendali.

PAIN POINTS

- Mancanza di tracciabilità nelle modifiche effettuate dagli sviluppatori.
- Tempi di attesa proibitivi per operazioni semplici.
- Piattaforme con una curva di apprendimento troppo ripida.

Figura 12: Roberto Esposito, Admin

3.6.2 Persona 2



*"Il codice è classe,
ogni bug è una lezione."*

About

- 22
- Bocconi, Milano
- Milano, Italia
- Maschio
- Junior Developer
- Single

Motivato Impulsivo Empatico

FAVORITE BRANDS



Francesco Sarto

BIO

Francesco Sarto è un giovane sviluppatore appena uscito dall'università con una laurea in Informatica. Appassionato di tecnologia fin da ragazzo, ha iniziato a programmare a 14 anni, creando piccoli progetti personali. Attualmente lavora come junior developer in una startup dove si occupa principalmente di sviluppo back-end con C++ e Java. È sempre alla ricerca di nuove sfide per migliorare le sue competenze e ama partecipare a hackathon e meetup locali. Preferisce lavorare in team e valorizza il feedback dei colleghi più esperti. Nel tempo libero contribuisce a progetti open source, ma è anche un appassionato di videogiochi, tant'è che frequenta fiere a tema, come il Milan Games Week.

GOALS

- Collaborare con il team tramite commenti.
- Monitorare lo stato delle issue assegnate in modo chiaro.
- Contribuire attivamente a progetti.

INTERESTS

- Gaming: appassionato di videogiochi platform e rogue-like.
- UI/UX design: non esperto, ma molto curioso in materia.
- Networking: partecipa regolarmente a meetup tech e conferenze.

NEEDS

- Disponibilità di chat integrata per fornire suggerimenti o richiedere chiarimenti.
- Ricevere notifiche sugli aggiornamenti delle issue assegnate.
- Interfaccia semplice e pronta all'uso.

PAIN POINTS

- Difficoltà nel filtrare e cercare le informazioni che ritiene rilevanti.
- Mancanza di shortcut per ottimizzare processi di routine.
- Strumenti che non garantiscono una comunicazione efficace con i Senior dev.

Figura 13: Francesco Sarto, Developer

3.7 Requisiti Non Funzionali

I Requisiti Non Funzionali definiscono le qualità di servizio, i vincoli architetturali e i criteri di qualità del sistema, in conformità allo standard **ISO/IEC 25010**.

[RNF-01] Usabilità e Accessibilità

L'interfaccia utente web deve essere intuitiva e reattiva. Tutte le operazioni principali devono essere eseguibili entro un massimo di 3 interazioni dall'interfaccia principale. Il sistema deve fornire un feedback visivo immediato per ogni operazione eseguita.

[RNF-02] Prestazioni e Scalabilità

Il sistema deve garantire un tempo di risposta inferiore a 2 secondi per le ricerche, i filtri e per il caricamento delle dashboard contenenti fino a 200 issue. Deve inoltre supportare l'accesso simultaneo di almeno 20 utenti concorrenti senza degrado significativo delle prestazioni.

[RNF-03] Sicurezza Applicativa

L'accesso al sistema richiede autenticazione basata su email e password. Le password devono essere memorizzate nel database in forma cifrata tramite algoritmi di hashing sicuri. Le comunicazioni client-server avvengono mediante API REST cifrate e il sistema adotta le linee guida OWASP contro attacchi XSS, CSRF e SQL Injection.

[RNF-04] Architettura Distribuita

Il codice sorgente deve rispettare la netta separazione delle responsabilità tra logica di visualizzazione e di business, organizzando il sistema per facilitare l'esecuzione di test unitari automatizzati e la scalabilità dei componenti.

[RNF-05] Compatibilità

L'applicazione web deve funzionare correttamente sui principali browser moderni come Chrome, Firefox e Safari. Il caricamento di allegati per le issue deve supportare i formati immagine più comuni, con un limite dimensionale di 5MB per singolo file.

3.8 Requisiti e Vincoli di Dominio

I Requisiti e Vincoli di Dominio comprendono le regole operative essenziali per il funzionamento della piattaforma e i vincoli normativi legati al mercato di distribuzione del software.

Regole Operative del Dominio

[RD-01] Cronologia delle Modifiche

Ogni bug deve possedere una cronologia che registri in modo immutabile l'identità dell'autore del cambiamento e il timestamp esatto, partendo obbligatoriamente dall'evento di creazione iniziale.

[RD-02] Classificazione e Stato Iniziale

Ogni issue deve essere classificata secondo una tipologia predefinita. Al momento della creazione, lo stato della segnalazione assume inderogabilmente il valore di partenza **todo**.

Vincoli Normativi di Dominio e Conformità Territoriale

[RD-03] Conformità alla Protezione Dati

Per consentire la distribuzione e la commercializzazione del software all'interno dell'Unione Europea, il sistema deve sottostare tassativamente al regolamento GDPR. La raccolta

dei dati personali avviene solo previo consenso esplicito, garantendo l'accesso, la rettifica e la cancellazione tramite il diritto all'oblio. Il mancato rispetto di tali direttive legali territoriali costituisce vincolo bloccante per la messa in produzione.

[RD-04] Unicità e Validazione delle Identità Utente

Ogni indirizzo e-mail registrato nel sistema deve essere univoco nell'ambito del database applicativo per prevenire duplicazioni di utenze e furti di identità. Eventuali collisioni devono impedire il completamento della registrazione fornendo immediata diagnostica.

3.9 Assunzioni e dipendenze

Lo sviluppo e il corretto funzionamento del sistema dipendono dalle seguenti condizioni preliminari:

- disponibilità di una connessione di rete stabile per il trasporto dei dati tra client e strato server;
- presenza di un'infrastruttura in grado di ospitare e orchestrare l'applicativo back-end e la sua persistenza;
- utilizzo vincolante di tool di progettazione CASE per la documentazione visiva;
- utilizzo di un sistema di versioning remoto per la storicizzazione del codice e degli artefatti;
- eventuali integrazioni con servizi terzi per l'erogazione delle comunicazioni email avverranno tramite API autenticate.

4 Documento di Design del Sistema

Il sistema *BugBoard26* è progettato secondo un'architettura multilivello client-server (Three-Tier), caratterizzata da una netta separazione delle responsabilità tra interfaccia utente, logica applicativa e persistenza.

- **Front-end (Presentation Layer):** un'applicazione web Single Page Application (SPA) in Angular, responsabile dell'interfaccia utente e della validazione client-side.
- **Back-end (Application Layer):** strutturato con approccio *Service-Based*, comprende due servizi Spring Boot autonomi che espongono API RESTful stateless per la gestione disgiunta dell'autenticazione e del dominio.
- **Persistence Layer:** database relazionale PostgreSQL per i dati strutturati, affiancato da un Docker Volume isolato per l'archiviazione dei contenuti multimediali.

4.1 Decomposizione del Sistema e Topologia dei Container

Il sistema si compone di quattro container isolati comunicanti in una *Docker Network* privata, orchestrati tramite Docker Compose. Per garantire il principio di *Defense in Depth*, ogni microservizio esegue i processi applicativi tramite un utente non-root (**appuser**), mitigando i rischi di *container breakout*.

- **Front-End (nginx):** Eroga i file statici della SPA fungendo da reverse proxy.
- **Auth Service (auth-service):** Dedicato alle operazioni di login, registrazione e firma dei token JWT. Interagisce col database con un'utenza dotata di privilegi CRUD esclusivi sulle tabelle **users** e **roles**.
- **Core Service (core-service):** Motore applicativo del tracciamento issue, ricerca, notifiche e audit. Interagisce col database tramite un'utenza con permessi CRUD sulle entità di dominio, ma vincolata in **sola lettura (READ-ONLY)** sull'anagrafica utenti, garantendo segregazione dei privilegi.
- **Database (postgresql):** Istanza isolata e inaccessibile dall'esterno, con dati salvaguardati su volumi persistenti.

4.2 Analisi delle Scelte Architetturelle e Trade-Off

Le decisioni ingegneristiche adottate derivano da una rigorosa analisi dei trade-off, mirata a bilanciare manutenibilità, scalabilità e complessità operativa.

4.2.1 Architettura Back-End, Front-End e Livello di Persistenza

Per il back-end, la scelta di un'architettura *Service-Based* scinde la gestione accessi dal core di business, offrendo i benefici dei microservizi senza l'eccessivo overhead infrastrutturale. Inoltre, si è preferito *Spring Boot* rispetto allo standard *Jakarta EE* per minimizzare la configurazione e facilitare il deployment containerizzato. Per il front-end, si è prediletta una SPA rispetto a un approccio multipagina tradizionale. Per il livello di persistenza, si è analizzata la strategia per le chiavi primarie, prediligendo **BIGSERIAL** rispetto a **SERIAL** e **UUID**. Le motivazioni sono sintetizzate nelle Tabelle 3, 4, 5 e 6.

Tabella 3: Trade-off Architetture Back-End

Criterio	Monolito	Service-Based (Scelta)	Microservizi Puri
Isolamento e Sicurezza	Basso (solo logico).	<i>Elevato</i> (Isolamento di processo e DB).	Massimo.
Integrità Transazionale	Garantita in memoria.	<i>Garantita</i> (Atomicità su Issue e Audit preservata).	Complessa (Saga Pattern).
Complessità Operativa	Minima.	<i>Contenuta</i> (Gestibile via Compose).	Molto Elevata (Service Mesh).

Tabella 4: Trade-off Framework Back-End

Criterio	Jakarta EE	Spring Boot (Scelta)
Configurazione e Setup	Prolissa; richiede Application Server esterni (WildFly, GlassFish).	<i>Minimale</i> ; Auto-Configuration e Web Server Tomcat embedded.
Integrazione Ecosystem	Richiede moduli e configurazioni manuali per Security e ORM.	<i>Nativa</i> ; integrazione immediata con Spring Security (JWT) e Spring Data JPA.
Containerizzazione	Peso dell'immagine elevato causa l'Application Server esterno.	<i>Ottimizzata</i> ; esecuzione tramite <code>.jar</code> eseguibile autonomo per Docker.

Tabella 5: Trade-off Architetture Front-End

Criterio	Multi-Page App (MPA)	Single-Page App (Angular - Scelta)
Esperienza Utente	Ricaricamento completo della pagina ad ogni azione.	<i>Fluida e reattiva</i> ; aggiornamento parziale del DOM tramite Data Binding.
Overhead di Rete	Elevato (trasferimento continuo di markup HTML).	<i>Minimo</i> (scambio esclusivo di payload JSON tramite API REST).
Gestione Sessione	Stateful sul server (Sessioni HTTP tradizionali).	<i>Stateless sul client</i> ; token JWT persistito in <code>localStorage</code> .

Tabella 6: Trade-off Identificativi Persistenza

Criterio	UUID (128-bit)	BIGSERIAL (64-bit - Scelta)
Impatto B-Tree	Frammentazione e page splitting elevati causa valori casuali.	<i>Inesistente</i> ; inserimento sequenziale sequenziato e B-Tree compatto.
Footprint Memoria	16 byte per record (raddoppia la RAM richiesta dagli indici).	<i>Ottimizzato</i> ; 8 byte per record, mapping 1:1 nativo col tipo Java <code>Long</code> .
Rischio Overflow	Assente.	<i>Praticamente nullo</i> (9×10^{18} valori, adeguato per audit log ad alto traffico).

4.2.2 Pattern Comunicativi e Strutturali

- **CQRS e Comunicazione (Sincrona vs Asincrona):** Il `core-service` applica il pattern CQRS. Le mutazioni di dominio impongono un tracciamento di audit sincrono e transazionale per garantire consistenza forte. Viceversa, l'erogazione delle notifiche adotta un approccio asincrono (*Event-Driven*), impedendo che ritardi nella messaggistica blocchino le transazioni core.
- **Autenticazione Stateless e Generazione Token (JWT e BCrypt):** L'autenticazione adotta un'architettura *stateless* tramite Spring Security e JSON Web Token. Le credenziali sono verificate dall'`auth-service` mediante hashing con BCrypt. Il token simmetrico firmato consente al `core-service` di convalidare le richieste autonomamente in memoria, senza effettuare chiamate via rete verso l'`auth-service`.
- **Storage Allegati (Strategy Pattern):** Per non degradare le performance del DB con oggetti BLOB, si è adottato lo *Strategy Pattern* (`StorageProvider`). L'implementazione attuale sfrutta un Docker Volume locale gestito da un `ResourceHandler` Spring, predisponendo il codice per future migrazioni complete in cloud (es. AWS S3 o Azure) senza impattare la business logic.
- **UI Pattern (Smart vs Dumb Components):** Nel front-end, la logica di rete e la gestione dello stato sono delegate esclusivamente agli *Smart Components*. Gli elementi grafici riutilizzabili (*Dumb Components*, es. modali o badge) operano unicamente tramite interazioni I/O in memoria (`@Input/@Output`), massimizzando testabilità e riuso.

4.3 Modellazione UML e Progettazione di Dettaglio

Vengono qui presentati i modelli strutturali e comportamentali UML 2.0. La trattazione segue un'organizzazione logica sequenziale: analizza inizialmente la topologia fisica dell'infrastruttura, approfondisce l'intera architettura statica e dinamica del Back-End, descrive la struttura modulare del Front-End, e si conclude con la modellazione relazionale dello Schema di Persistenza del Database.

4.3.1 Deployment Diagram: Topologia e Hardening Infrastrutturale

Il diagramma (Figura 14) traduce la topologia di rete. La convalida dei token JWT nel `core-service` avviene mediante decifratura asimmetrica locale, evitando colli di bottiglia verso l'`auth-service`. L'assenza di accessi root nei container concretizza i requisiti di irrobustimento del sistema.

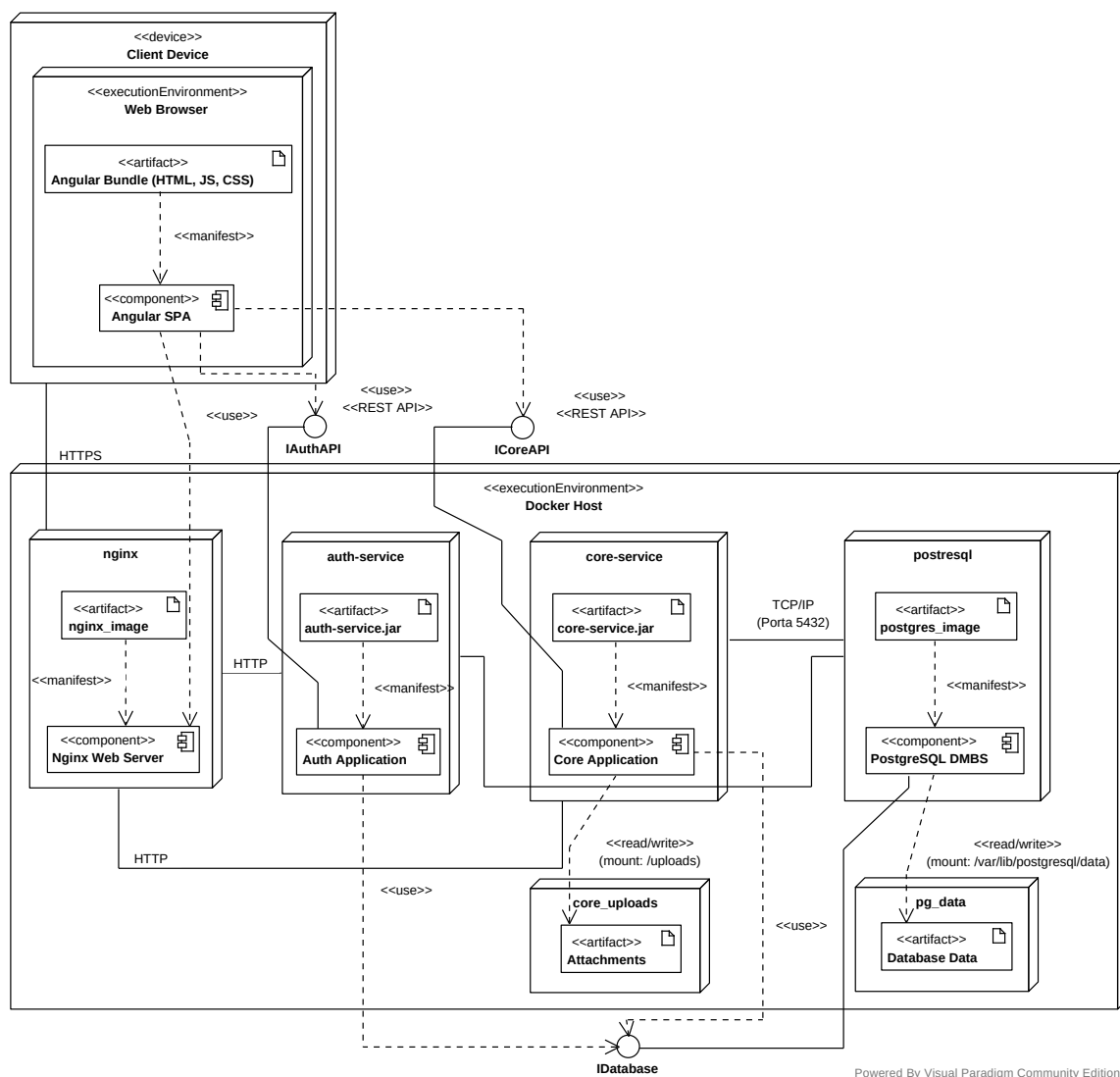


Figura 14: Deployment Diagram - Topologia dell'Infrastruttura BugBoard26

4.3.2 Class Diagram: auth-service

Il package `com.bugboard26.auth` (Figura 15) modella il dominio di sicurezza, incapsulando la persistenza delle utenze, la validazione delle credenziali e l'integrazione nativa con i filtri di Spring Security.

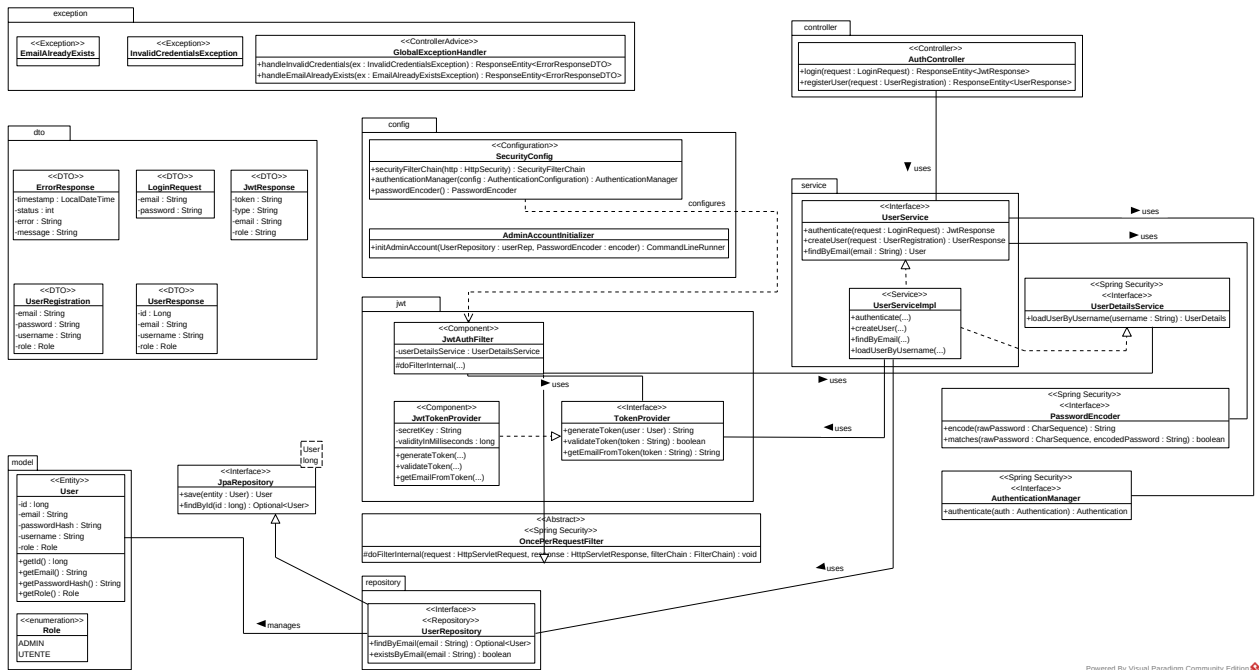


Figura 15: Class Diagram - Authentication Subsystem

4.3.3 Component Diagram: Architettura del core-service

Il diagramma (Figura 16) evidenzia l'assemblaggio dei quattro sottosistemi logici. Il disaccoppiamento è garantito dall'utilizzo di interfacce fornite (lollipop) e richieste (socket) nel pieno rispetto dell'Interface Segregation Principle.

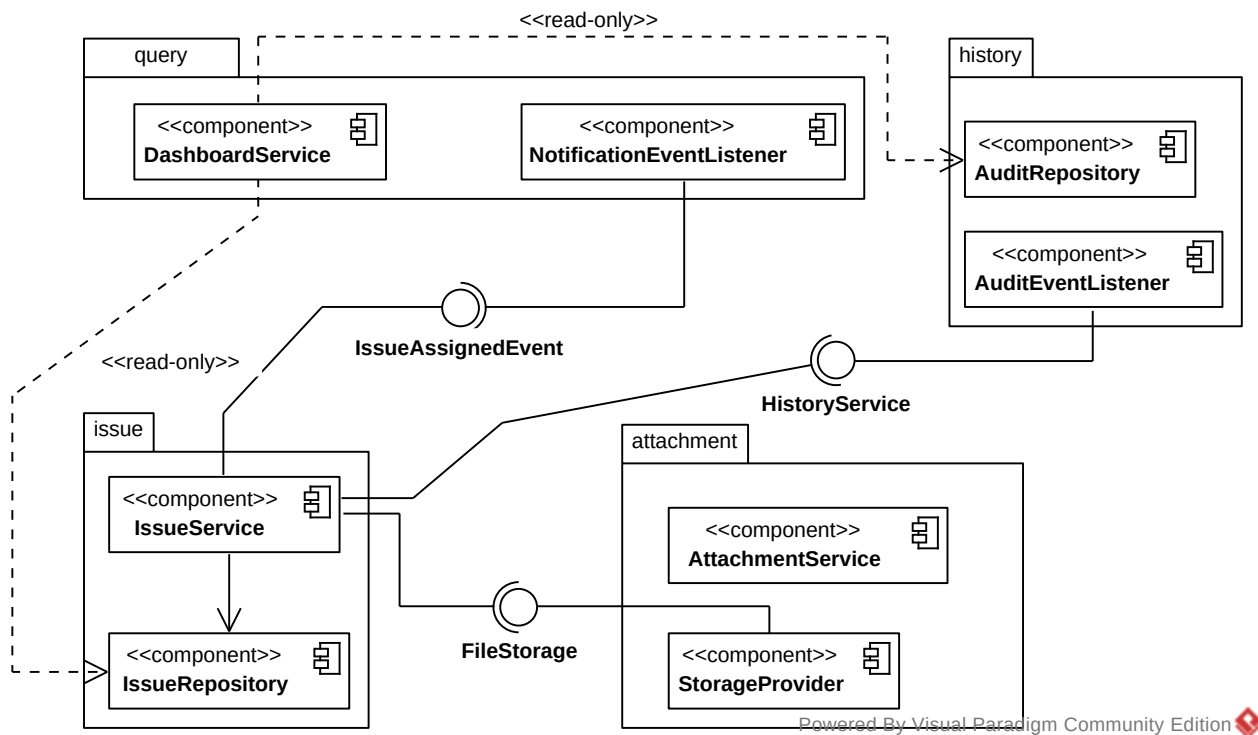


Figura 16: Component Diagram - Architettura Interna del Core Service

Query & View Subsystem Rappresenta il livello di sola lettura. Previene l'uso di metodi mutazionali estendendo interfacce `ReadOnlyRepository`. Ottimizza le interrogazioni per i filtri di dashboard e la risoluzione asincrona degli eventi di notifica.

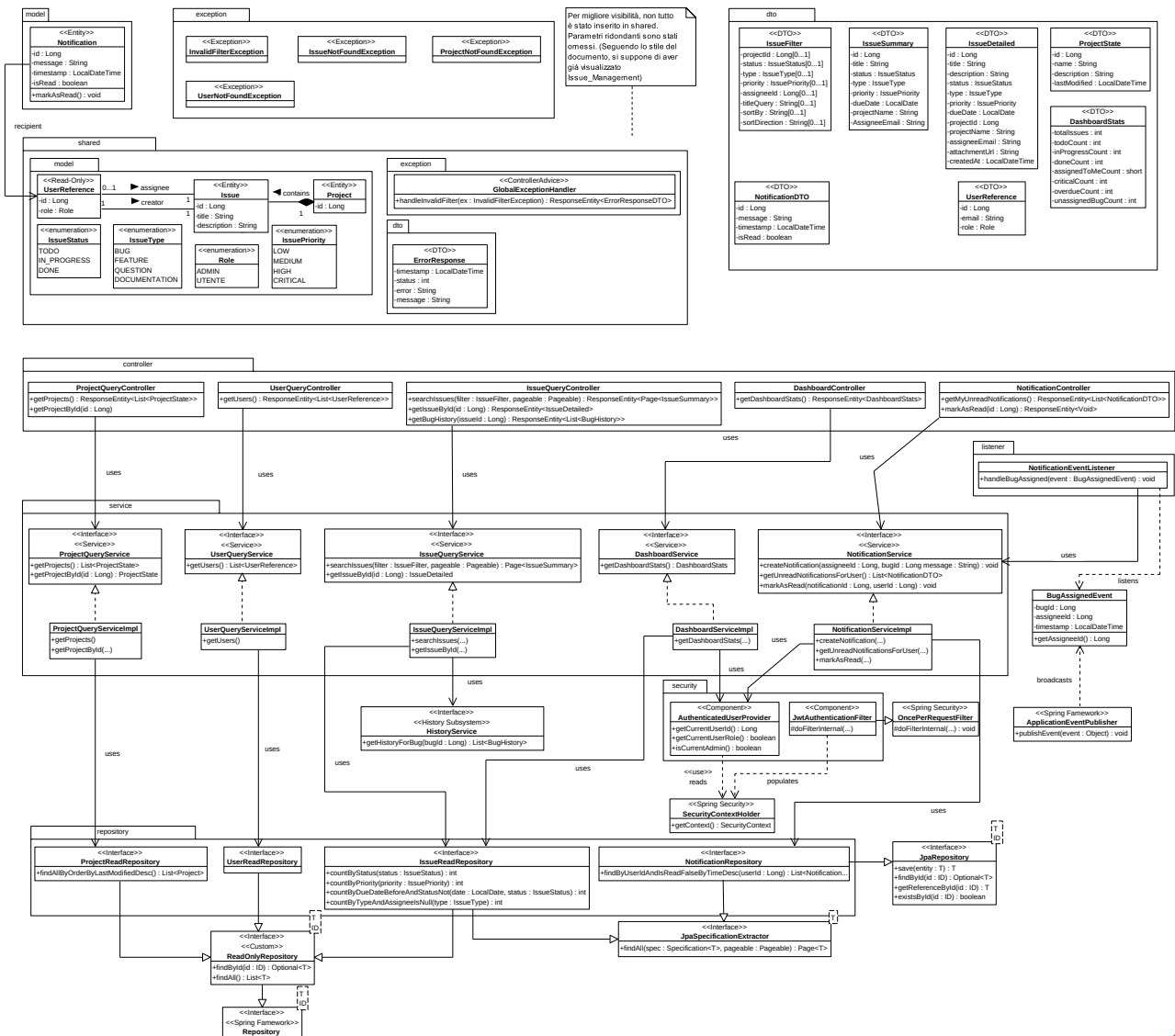


Figura 18: Class Diagram - Query & View Subsystem

History Subsystem Modella il tracciamento di audit. Garantisce l'immutabilità dei record storici classificando le transazioni operative tramite l'enumerazione **AuditAction**.

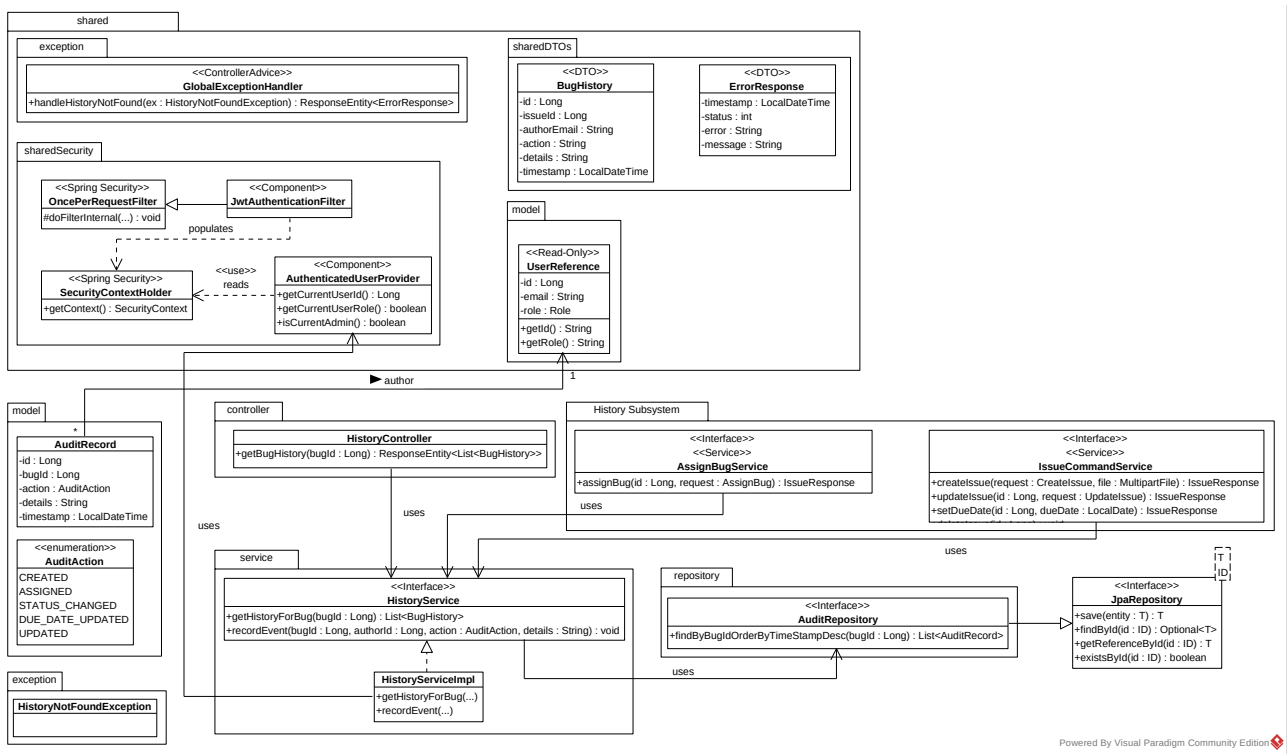
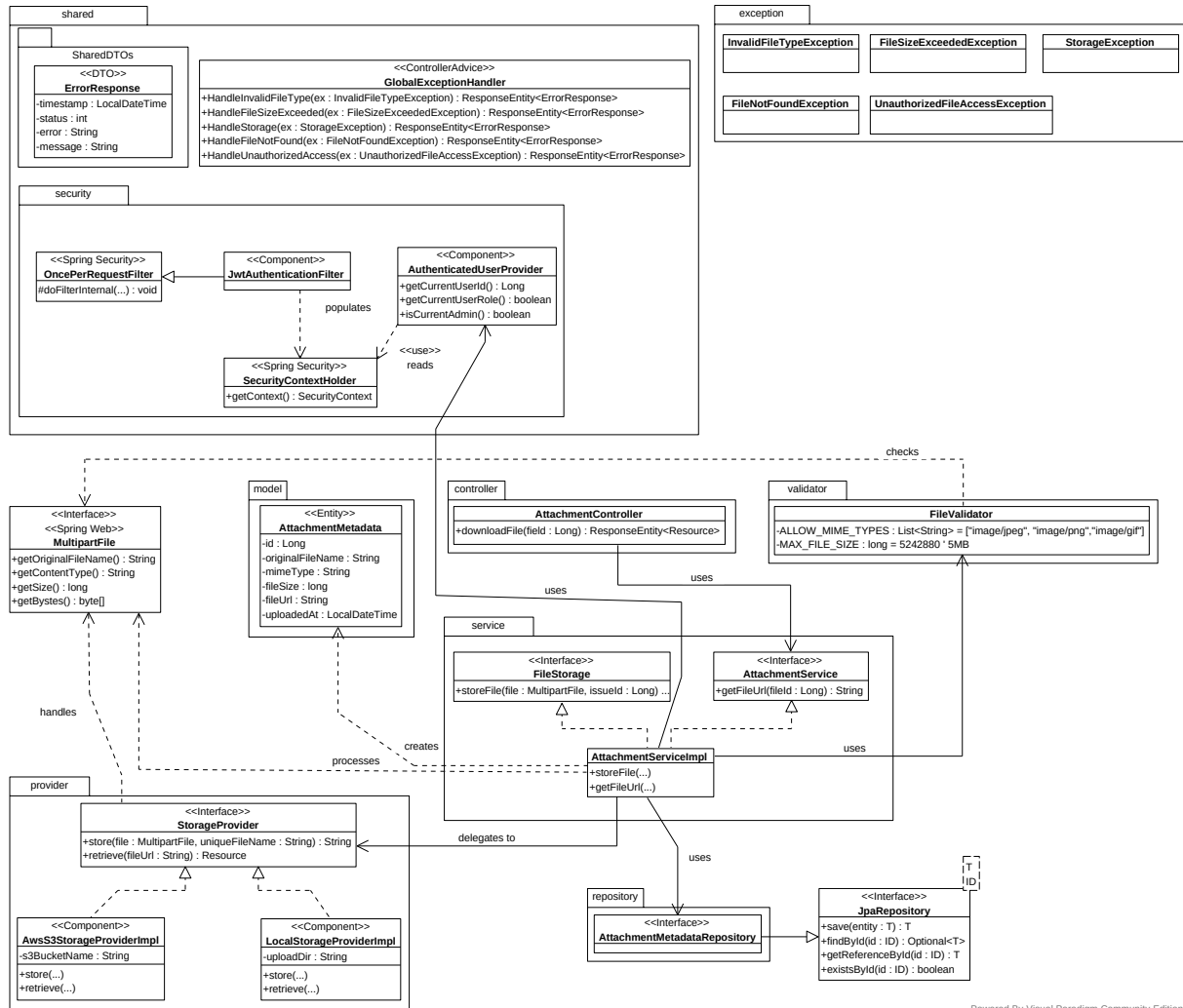


Figura 19: Class Diagram - History Subsystem

Attachment Subsystem Disaccoppia la persistenza dei file binari. Conserva a database esclusivamente i metadati spaziali (**AttachmentMetadata**), delegando il salvataggio fisico all'implementazione concreta definita dallo Strategy Pattern.



Powered By Visual Paradigm Community Edition

Figura 20: Class Diagram - Attachment Subsystem

4.3.5 Modellazione Comportamentale e Dinamica del Back-End

Sequence Diagram: Creazione della Issue Modella la propagazione temporale dei messaggi (Figura 21). Evidenzia la gestione anticipata delle eccezioni di validazione tramite frammenti **break** e documenta l'invocazione transazionale per la storicizzazione, garantendo l'atomicità delle operazioni.

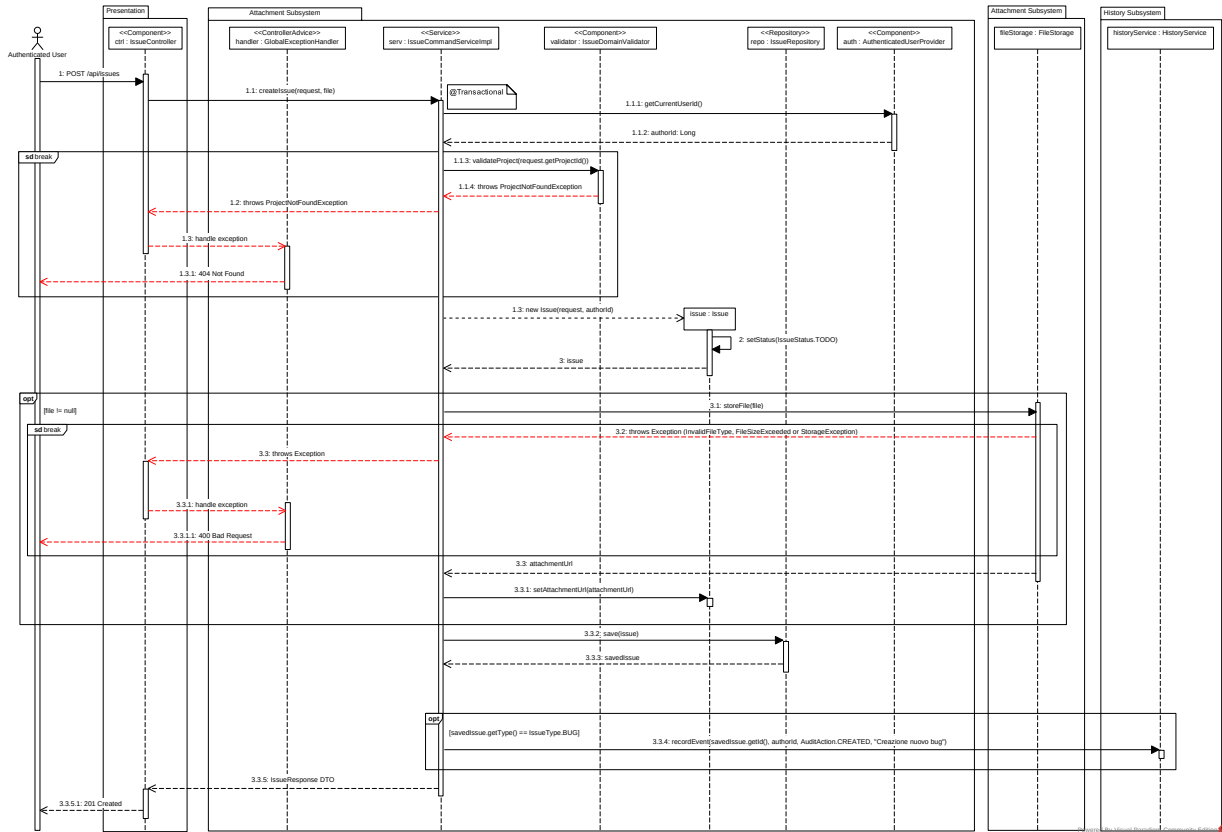


Figura 21: Sequence Diagram - Transazione di Creazione Issue

Statechart Diagram: Ciclo di Vita della Issue Definisce l'evoluzione comportamentale dell'entità (Figura 22). Il design centralizza le operazioni di aggiornamento parziale nel super-stato **OPEN** ed impiega espressioni booleane (*guards*) collegate all'**AccessControlValidator** per garantire il rispetto dei privilegi sui cambi di stato.

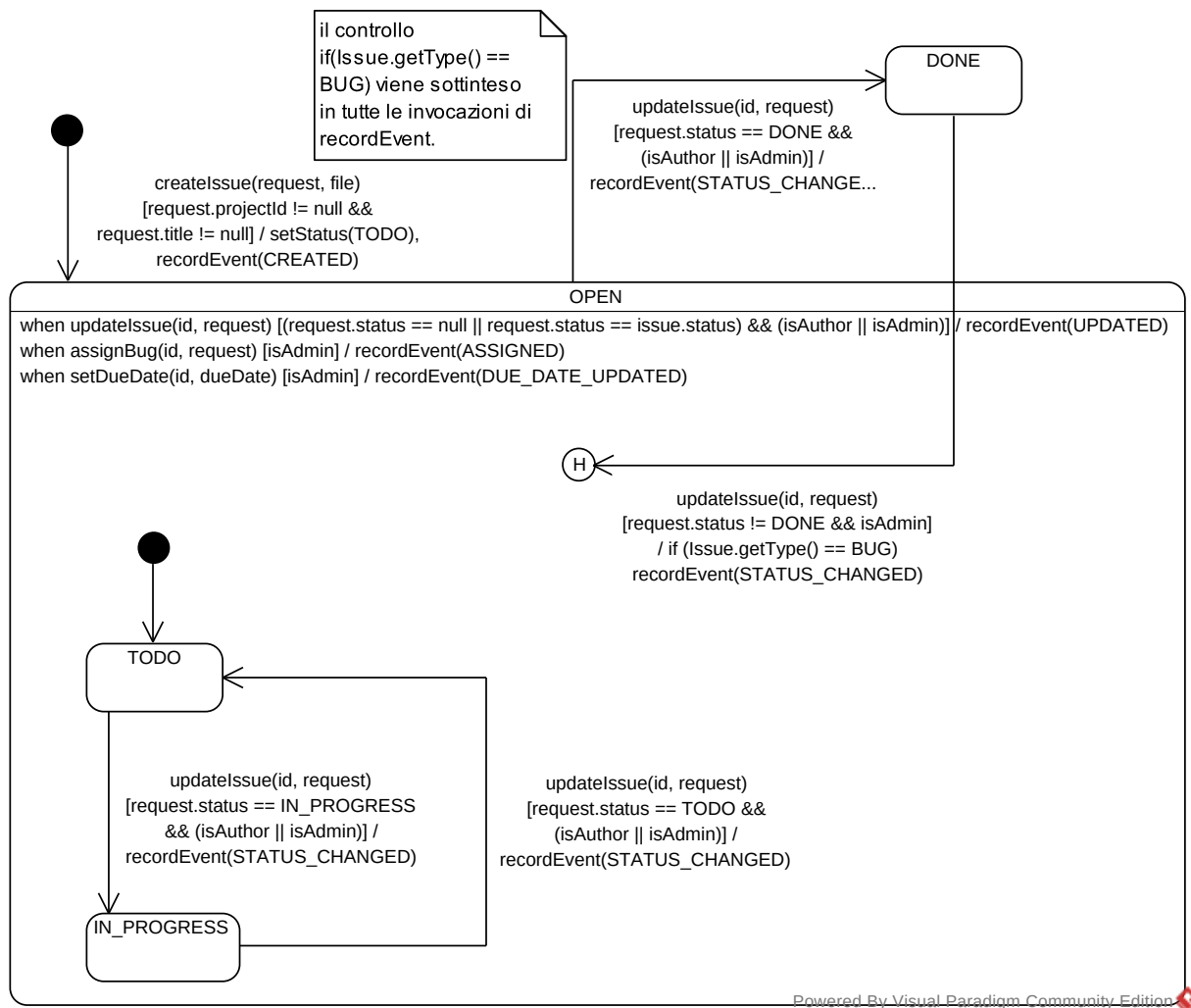


Figura 22: Statechart Diagram - Ciclo di Vita e Transizioni

Activity Diagram: Assegnazione Bug e Notifica Descrive l'orchestrazione asincrona in ottica CQRS (Figura 23). Attraverso i *Decision Nodes*, verifica l'autorizzazione di sistema e la validità del dominio, per poi biforcare l'esecuzione (*fork*): consolida l'assegnazione nel database primario e parallelamente pubblica l'evento per la generazione della notifica utente.

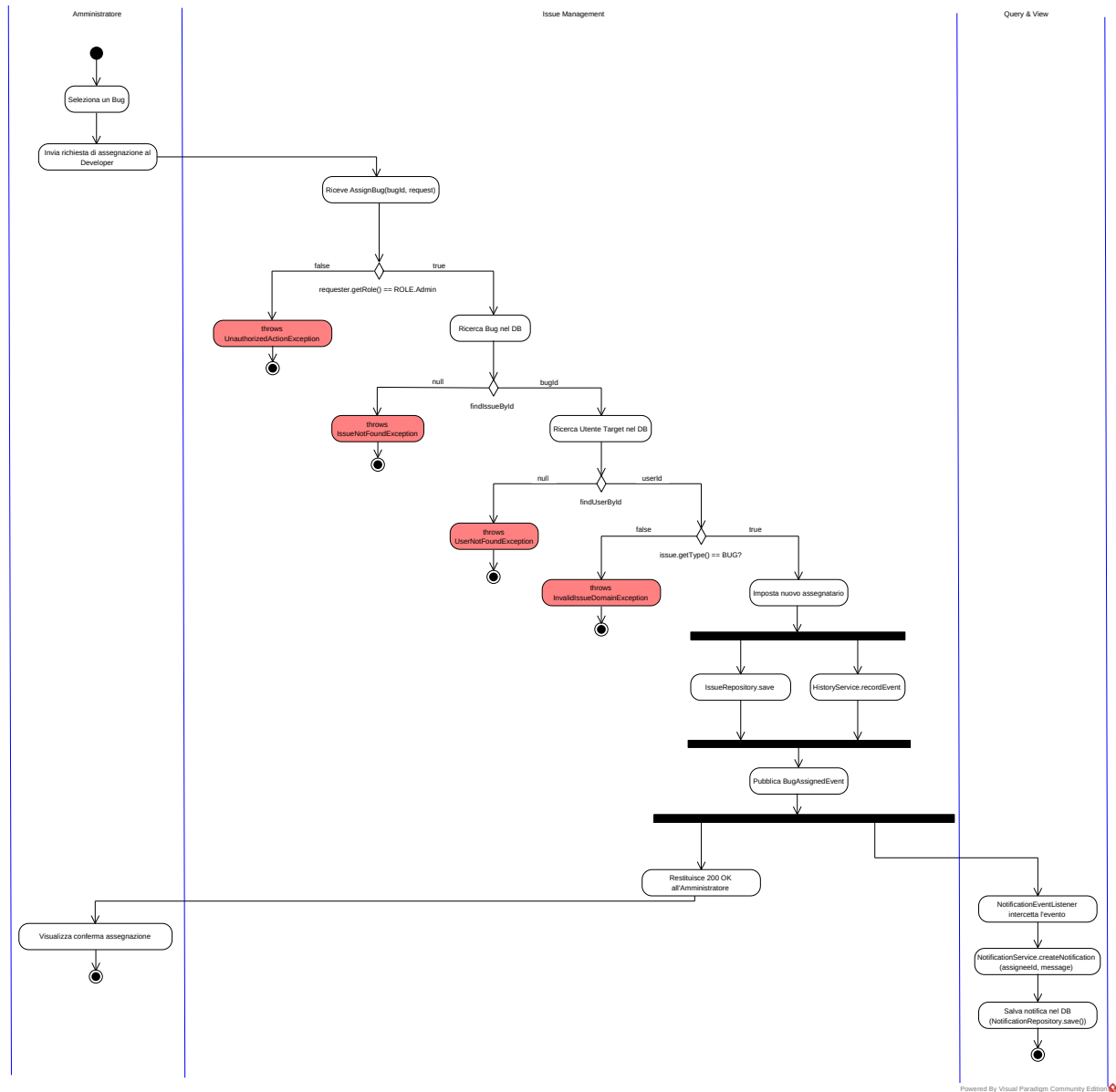


Figura 23: Activity Diagram - Processo di Assegnazione e Notifica Asincrona

4.3.6 Component Diagram: Moduli Front-End Angular

L'architettura della SPA (Figura 24) è strutturata in quattro moduli coesi per ottimizzare il rendering mediante *Lazy Loading*. Riflette in modo rigoroso l'API Contract con il back-end, implementando interfacce DTO TypeScript validate a tempo di compilazione.

- **CoreAuthModule:** Modulo caricato all'avvio. Raggruppa i servizi di sessione e utilizza le funzioni `AuthGuard` e `AdminGuard` per la protezione RBAC delle rotte. Intercetta globalmente la pipeline HTTP (`JwtInterceptor`, `ErrorInterceptor`).
- **IssueModule:** Modulo di feature per l'inserimento e modifica dei dati. Impiega form reattive e DTO speculari al Command Layer.
- **DashboardQueryModule:** Interfaccia di lettura e ricerca. Gestisce filtri dinamici, tabelle e audit trail mappando i payload sulle interfacce esposte dal Query Layer.
- **SharedModule:** Incapsula i *Presentational Components* (modali, badge) trasversali all'applicazione.

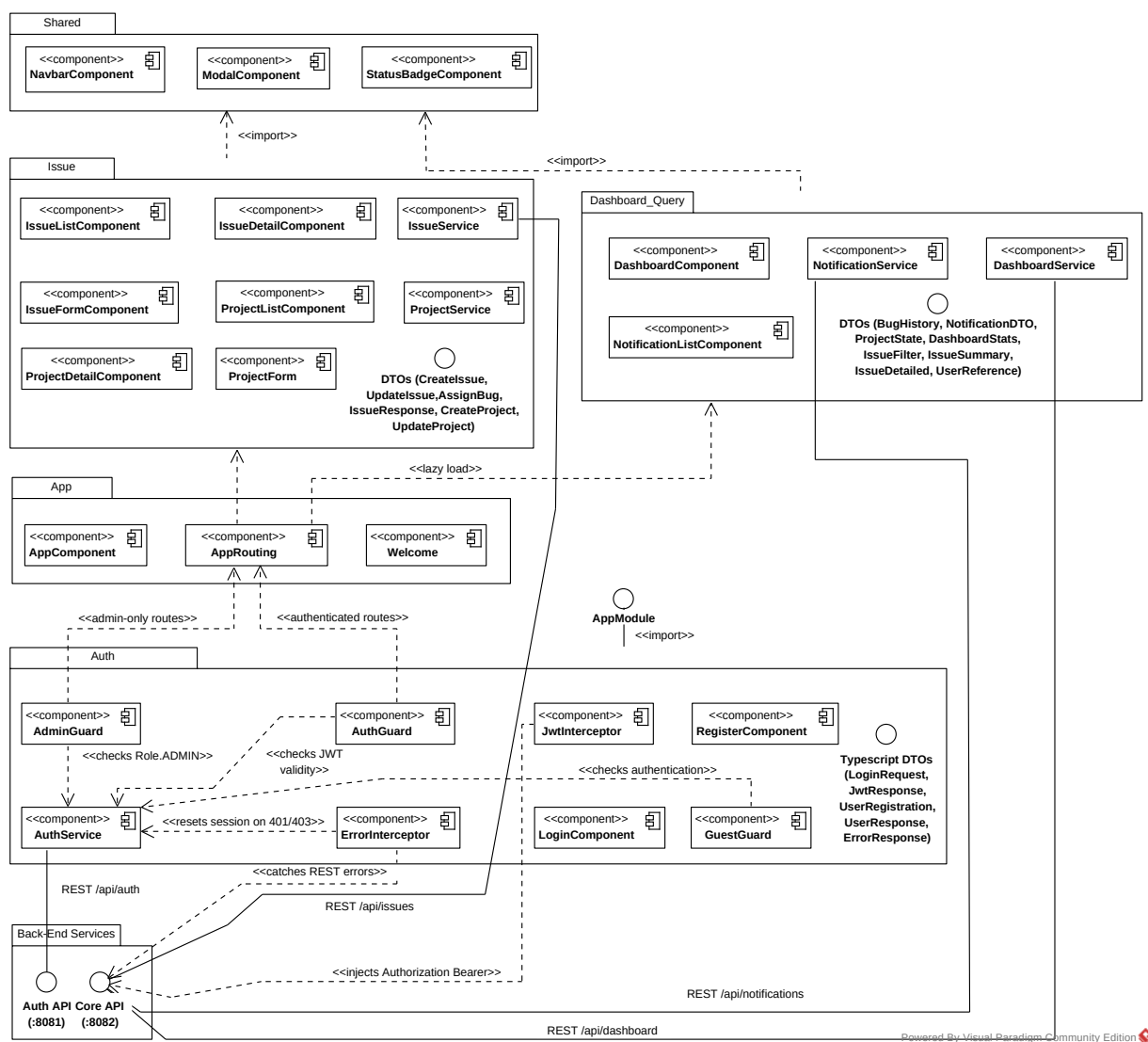


Figura 24: Component Diagram - Moduli dell'Applicazione Front-End Angular

4.3.7 Schema per la Persistenza Dati e Diagramma ER

La persistenza dei dati del sistema *BugBoard26* è affidata al DBMS relazionale PostgreSQL 15, containerizzato tramite Docker e reso accessibile esclusivamente all'interno della rete privata orchestrata da Docker Compose.

L'interfacciamento tra il codice Java e il database relazionale è realizzato tramite **Spring Data JPA** con **Hibernate** quale provider ORM sottostante. La definizione dello schema avviene in modo dichiarativo mediante le annotazioni JPA sulle entità di dominio:

- Le enumerazioni applicative (`IssueType`, `IssueStatus`, `IssuePriority`, `Role`, `AuditAction`) vengono convertite in colonne `VARCHAR` gestendole in Java mediante l'annotazione `@Enumerated(EnumType.STRING)`;
- In ambiente di runtime, la configurazione della proprietà `spring.jpa.hibernate.ddl-auto=update` consente ad Hibernate di sincronizzare ed aggiornare automaticamente la struttura fisica delle tabelle all'avvio dei microservizi, garantendo la coerenza tra il modello ad oggetti e lo schema relazionale.

Segregazione delle Viste e Hardening a Livello di DBMS In conformità con il principio del minimo privilegio (*Principle of Least Privilege*), la separazione tra `auth-service` e `core-service` è garantita a livello fisicamente strutturale mediante due ruoli di connessione PostgreSQL distinti ed una vista SQL dedicata (`user_reference`).

Tale architettura assicura che il `core-service` possa accedere all'anagrafica utenti esclusivamente in sola lettura ed escludendo fisicamente l'hash delle credenziali di accesso. La configurazione dei privilegi sul DBMS (Data Control Language) è definita come segue:

```
1  -- Creazione del namespace dedicato all'applicazione
2  CREATE SCHEMA IF NOT EXISTS bugboard;
3
4  -- 1. Ruolo Auth Service: Privilegi CRUD completi sulla gestione utenze
5  CREATE USER auth_service_user WITH PASSWORD 'auth_sec_pwd';
6  GRANT USAGE ON SCHEMA bugboard TO auth_service_user;
7  GRANT ALL PRIVILEGES ON TABLE bugboard.users TO auth_service_user;
8
9  -- 2. Ruolo Core Service: CRUD sulle entita di dominio, READ-ONLY sulla
   View Utenti
10 CREATE USER core_service_user WITH PASSWORD 'core_sec_pwd';
11 GRANT USAGE ON SCHEMA bugboard TO core_service_user;
12 GRANT ALL PRIVILEGES ON TABLE bugboard.issues, bugboard.projects, bugboard
   .attachments, bugboard.bug_history, bugboard.notifications TO
   core_service_user;
13 GRANT SELECT ON bugboard.user_reference TO core_service_user;
```

Struttura delle Tabelle e Vincoli d'Integrità Referenziale Il Diagramma Entità-Relazione (Figura 25) illustra il modello logico relazionale del sistema, definendo le entità di dominio, le chiavi primarie, le relazioni e le relative molteplicità.

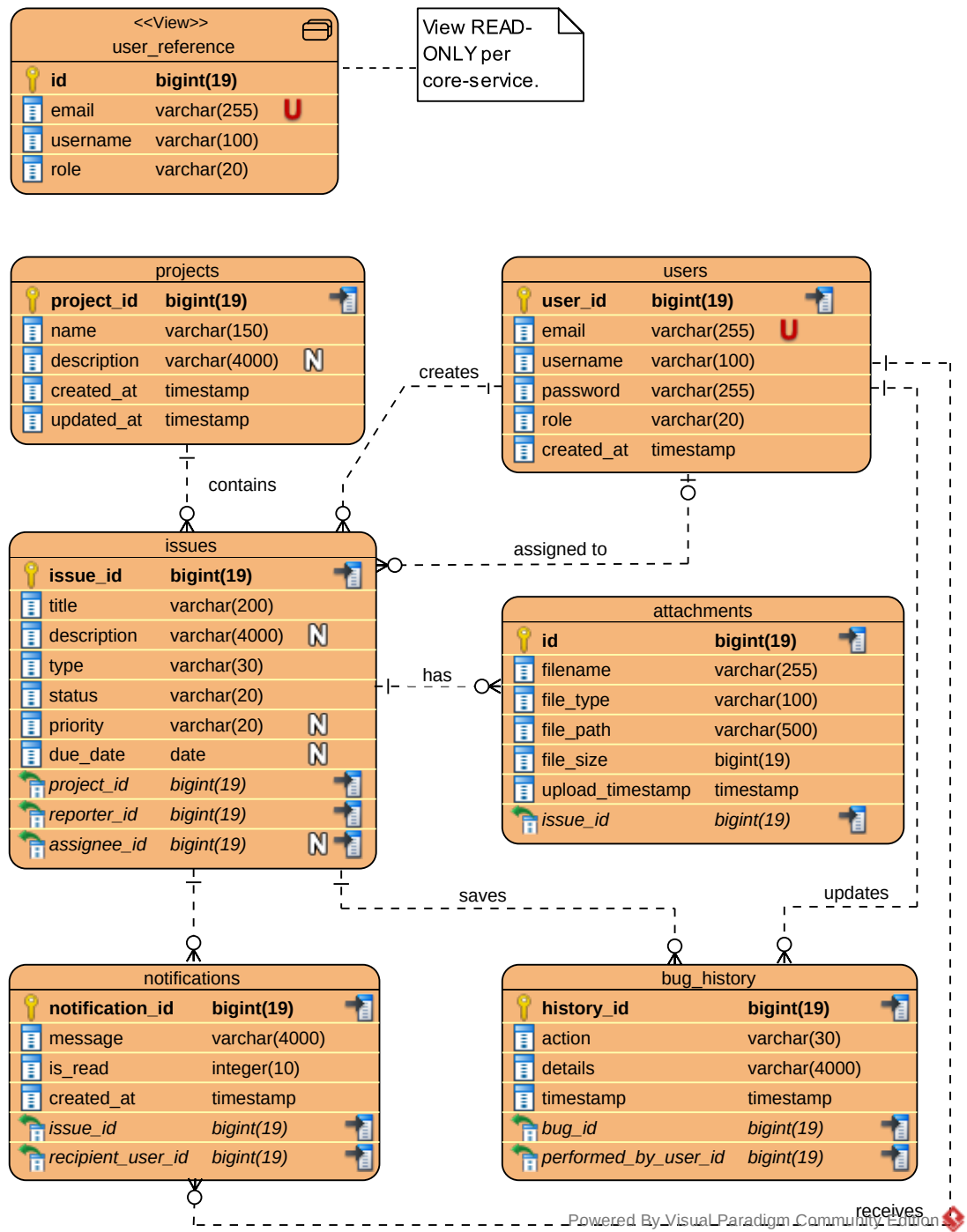


Figura 25: Diagramma Entità-Relazione dello Schema di Persistenza PostgreSQL

A completamento della modellazione grafica, di seguito è riportato lo schema fisico DDL (*Data Definition Language*). Il codice definisce in un unico script sequenziale la creazione delle tabelle, della vista di proiezioni `user_reference`, dei vincoli di integrità referenziale (`ON DELETE CASCADE/SET NULL`) e degli indici B-Tree composti per l'ottimizzazione delle query.

```

1  -- =====
2  -- Namespace: bugboard
3  -- =====
4
5  -- 1. Creazione Schema e Ruoli di Sicurezza
6  CREATE SCHEMA IF NOT EXISTS bugboard;
7
8  CREATE USER auth_service_user WITH PASSWORD 'auth_sec_pwd';
9  GRANT USAGE ON SCHEMA bugboard TO auth_service_user;
10
11 CREATE USER core_service_user WITH PASSWORD 'core_sec_pwd';
12 GRANT USAGE ON SCHEMA bugboard TO core_service_user;
13
14 SET search_path TO bugboard;
15
16 -- 2. Tabella Utente (Auth Service)
17 CREATE TABLE users (
18     id BIGSERIAL PRIMARY KEY,
19     email VARCHAR(255) NOT NULL UNIQUE,
20     username VARCHAR(100) NOT NULL,
21     password VARCHAR(255) NOT NULL,
22     role VARCHAR(20) NOT NULL,
23     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
24 );
25
26 -- Permessi su users
27 GRANT ALL PRIVILEGES ON TABLE users TO auth_service_user;
28 GRANT USAGE, SELECT ON SEQUENCE users_id_seq TO auth_service_user;
29
30 -- Vista Read-Only per il Core Service
31 CREATE VIEW user_reference AS SELECT id, email, username, role FROM users;
32 GRANT SELECT ON user_reference TO core_service_user;
33
34 -- 3. Tabella Progetti
35 CREATE TABLE projects (
36     id BIGSERIAL PRIMARY KEY,
37     name VARCHAR(150) NOT NULL,
38     description TEXT,
39     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
40     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
41 );
42
43 -- 4. Tabella Issue
44 CREATE TABLE issues (
45     id BIGSERIAL PRIMARY KEY,
46     title VARCHAR(200) NOT NULL,
47     description TEXT,
48     type VARCHAR(30) NOT NULL,
49     status VARCHAR(20) NOT NULL,
50     priority VARCHAR(20),
51     due_date DATE,
52     attachment_url VARCHAR(500),
53     project_id BIGINT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
54     reporter_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
55     assignee_id BIGINT REFERENCES users(id) ON DELETE SET NULL,

```

```

56     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
57     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
58 );
59
60 -- 5. Tabella Allegati
61 CREATE TABLE attachments (
62     id BIGSERIAL PRIMARY KEY,
63     filename VARCHAR(255) NOT NULL,
64     file_type VARCHAR(100) NOT NULL,
65     file_path VARCHAR(500) NOT NULL,
66     file_size BIGINT NOT NULL,
67     upload_timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
68     issue_id BIGINT NOT NULL REFERENCES issues(id) ON DELETE CASCADE
69 );
70
71 -- 6. Tabella Audit / History
72 CREATE TABLE bug_history (
73     id BIGSERIAL PRIMARY KEY,
74     bug_id BIGINT NOT NULL REFERENCES issues(id) ON DELETE CASCADE,
75     action VARCHAR(30) NOT NULL,
76     details TEXT,
77     timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
78     performed_by_user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE
79     RESTRICT
80 );
81
82 -- 7. Tabella Notifiche
83 CREATE TABLE notifications (
84     id BIGSERIAL PRIMARY KEY,
85     recipient_user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE
86     CASCADE,
87     bug_id BIGINT NOT NULL REFERENCES issues(id) ON DELETE CASCADE,
88     message TEXT NOT NULL,
89     is_read BOOLEAN NOT NULL DEFAULT FALSE,
90     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
91 );
92
93 -- Permessi su dominio applicativo al Core Service
94 GRANT ALL PRIVILEGES ON TABLE projects, issues, attachments, bug_history,
95     notifications TO core_service_user;
96 GRANT USAGE, SELECT ON SEQUENCE
97     projects_id_seq,
98     issues_id_seq,
99     attachments_id_seq,
100     bug_history_id_seq,
101     notifications_id_seq
102 TO core_service_user;
103
104 -- 8. Indici per Performance
105 CREATE INDEX idx_issues_project_status ON issues(project_id, status);
106 CREATE INDEX idx_issues_assignee ON issues(assignee_id);
107 CREATE INDEX idx_issues_type ON issues(type);
108 CREATE INDEX idx_bug_history_bug_id ON bug_history(bug_id);
109 CREATE INDEX idx_notifications_recipient_read ON notifications(
110     recipient_user_id, is_read);

```

5 Containerizzazione e Infrastruttura di Deployment

5.1 Build delle Immagini: Multi-Stage e Sicurezza

Per la generazione delle immagini applicative dei microservizi (`auth-service` e `core-service`) è stata adottata la strategia operativa della *Multi-Stage Build*. Questa tecnica permette di separare l'ambiente di compilazione da quello di esecuzione, riducendo drasticamente le dimensioni dell'immagine finale e la superficie di attacco.

Di seguito è riportato il Dockerfile utilizzato per il `core-service` (analogo a quello dell'`auth-service`):

```
1 # =====
2 # STAGE 1: BUILD
3 # =====
4 FROM amazoncorretto:21-alpine AS builder
5 WORKDIR /app
6
7 # Download delle dipendenze offline per ottimizzare il layer di cache
8 # Docker
9 COPY .mvn/ .mvn/
10 COPY mvnw pom.xml ./
11 RUN chmod +x ./mvnw
12 RUN ./mvnw dependency:go-offline
13
14 # Copia e compilazione del codice sorgente
15 COPY src ./src
16 RUN ./mvnw clean package -DskipTests
17
18 # =====
19 # STAGE 2: RUNTIME
20 # =====
21 FROM amazoncorretto:21-alpine
22 WORKDIR /app
23
24 # Creazione utente non-root per mitigare i rischi di container breakout
25 RUN addgroup -S appgroup && adduser -S appuser -G appgroup
26 USER appuser
27
28 # Copia del solo artefatto compilato dallo stage precedente
29 COPY --from=builder /app/target/*.jar app.jar
30
31 EXPOSE 8080
32 ENTRYPOINT ["java", "-jar", "app.jar"]
```

L'uso dell'immagine `amazoncorretto:21-alpine` garantisce un footprint di memoria minimo, ideale per i microservizi Java. Inoltre, la direttiva `USER appuser` implementa a livello infrastrutturale il *Principle of Least Privilege*, mitigando le minacce di *container breakout* evidenziate in fase di design.

Di seguito è riportato il `Dockerfile` utilizzato per il `front-end`. Utilizziamo `node 24-alpine` per istanziare il codice Angular nella corretta versione di progetto, ed `nginx:alpine` per il reverse proxy necessario per far operare sulla rete il nostro front-end.

```
1  # =====
2  # STAGE 1: BUILD (Angular)
3  # =====
4  FROM node:24-alpine AS builder
5  WORKDIR /app
6
7  # Copia i file di dipendenza e installa
8  COPY package*.json ./
9  RUN npm ci
10
11 # Copia il resto del codice e compila l'app per la produzione
12 COPY . .
13 RUN npm run build --configuration=production
14
15 # =====
16 # STAGE 2: RUNTIME (Nginx)
17 # =====
18 FROM nginx:alpine
19
20 # Rimuove la configurazione di default di Nginx
21 RUN rm /etc/nginx/conf.d/default.conf
22
23 # Copia la configurazione personalizzata creata nello Step 1
24 COPY nginx.conf /etc/nginx/conf.d/
25
26 # Copia l'output della build di Angular nella cartella di Nginx
27 COPY --from=builder /app/dist/front-end/browser /usr/share/nginx/html
28
29 EXPOSE 80
30
31 CMD ["nginx", "-g", "daemon off;"]
```

5.2 Orchestrazione dei Servizi: Docker Compose

L'intera topologia di rete, i volumi persistenti e le dipendenze di avvio sono definite in modo dichiarativo nel file `docker-compose.yaml`. L'architettura è configurata per orchestrare automaticamente il database PostgreSQL e i microservizi (back-end e front-end) in una Virtual Network privata.

```

1  services
2    # 1. DATABASE POSTGRESQL 15
3    postgresql
4      image postgres15-alpine
5      container_name bugboard_postgres
6      environment
7        POSTGRES_DB bugboard_db
8        POSTGRES_USER postgres
9        POSTGRES_PASSWORD root
10     ports
11       - "5432:5432"
12     volumes
13       - pgdata/var/lib/postgresql/data
14       - ./database/init.sql/docker-entrypoint-initdb.d/init.sql
15     networks
16       - bugboard_network
17     restart unless-stopped
18     healthcheck
19       test ["CMD-SHELL", "pg_isready -U postgres -d bugboard_db"]
20       interval 5s
21       timeout 5s
22       retries 5
23
24   # 2. AUTH SERVICE
25   auth-service
26     build
27       context ./back-end/auth-service
28       dockerfile Dockerfile
29     container_name bugboard_auth_service
30     ports
31       - "8081:8081"
32     environment
33       - SPRING_DATASOURCE_URL=jdbc:postgresql://postgresql5432/bugboard_db?
34         currentSchema=bugboard
35       - ADMIN_EMAIL=${ADMIN_EMAIL}
36       - ADMIN_USERNAME=${ADMIN_USERNAME}
37       - ADMIN_PASSWORD=${ADMIN_PASSWORD}
38     depends_on
39       postgresql
40       condition service_healthy
41     networks
42       - bugboard_network
43     restart on-failure
44
45   # 3. CORE SERVICE
46   core-service
47     build
48       context ./back-end/core-service
49       dockerfile Dockerfile
50     container_name bugboard_core_service
51     ports
52       - "8080:8080"
53     environment

```

```

53     - SPRING_DATASOURCE_URL=jdbcpostgresql//postgresql5432/bugboard_db?
currentSchema=bugboard
54     - SPRING_DATASOURCE_USERNAME=core_service_user
55     - SPRING_DATASOURCE_PASSWORD=core_sec_pwd
56     volumes
57         # Evitiamo la perdita di allegati e file caricati dagli utenti,
montando la cartella core_uploads del container in una cartella locale
58         - ./back-end/core-service/core_uploads/app/core_uploads
59     depends_on
60         postgresql
61         condition service_healthy
62     networks
63         - bugboard_network
64     restart on-failure
65
66 # 4. FRONT-END (NGINX)
67 front-end
68     build
69         context ./front-end
70         dockerfile Dockerfile
71     container_name bugboard_frontend
72     ports
73         - "80:80"
74     depends_on
75         - auth-service
76         - core-service
77     networks
78         - bugboard_network
79     restart unless-stopped
80
81 networks
82     bugboard_network
83     driver bridge
84
85 volumes
86     pgdata

```

5.3 Gestione dei Volumi e Sincronizzazione dell'Avvio

L'orchestrazione definita nel `docker-compose.yaml` concretizza due aspetti cruciali del design architetturale:

- **Healthcheck e Sequenza di Avvio Corretta:** L'attributo `depends_on` configurato con `condition: service_healthy` all'interno dei microservizi vincola il loro avvio all'effettiva disponibilità transazionale del database PostgreSQL. Questo elimina in modo strutturale i loop di crash dovuti a tentativi di connessione prematuri del framework Spring Boot.
- **Segregazione della Persistenza:** I dati relazionali sono salvaguardati nel Docker Volume gestito `pgdata`. Parallelamente, i file multimediali caricati dagli utenti vengono persistiti mappando la directory `./core_uploads` al percorso interno `/app/core_uploads` del container `core-service`. Questo isola fisicamente i BLOB dal database PostgreSQL e previene la perdita degli allegati al riavvio del container.

6 Versioning e Gestione del Codice Sorgente

6.1 Sistema di Version Control

Il progetto BugBoard26 utilizza **Git** come sistema di version control distribuito, ospitato su un repository **GitHub**. La scelta di Git è motivata dalla sua diffusione universale nell'ecosistema di sviluppo software, dall'affidabilità nel tracciamento delle modifiche e dalla sua totale integrazione con i principali ambienti di sviluppo e pipeline di automazione.

Il repository è stato organizzato secondo il pattern architetturale del **Monorepo**: contiene al suo interno l'intera codebase del sistema, separando logicamente i domini (come il frontend Angular, l'*auth-service* e il *core-service* del backend) in directory distinte, facilitando la gestione unificata delle dipendenze e della documentazione.

Il repository ufficiale del progetto è visionabile al seguente [link](#).

6.2 Strategia di Sviluppo e Integrazione

Coerentemente con le dimensioni del team e con le best practice per le architetture a microservizi, è stata adottata una strategia di branching semplificata basata sul **Trunk-Based Development**.

Tutto lo sviluppo converge direttamente sul branch **main**, che rappresenta in ogni momento lo stato stabile, verificato e rilasciabile del sistema. La politica adottata prevede una regola fondamentale: *nessun codice corrotto deve mai raggiungere il branch principale*.

Per far rispettare questo vincolo, il flusso di lavoro è strettamente supportato dalla **Continuous Integration (CI)**.

6.3 Statistiche del Repository

Di seguito vengono riportate le metriche principali estratte dal repository GitHub, che testimoniano l'andamento e la frequenza dello sviluppo del progetto BugBoard26:

- **Numero totale di commit:** 130
- **Linee di codice scritte (LOC):** 6,000~



Figura 26: Estratto della cronologia dei commit sul branch main di BugBoard26

6.4 Evidenza della Cronologia Git

Al fine di dimostrare l'applicazione della strategia descritta, di seguito viene riportato un estratto significativo della cronologia dei commit effettuati sul branch **main**. È possibile notare come i messaggi di commit siano stati formattati in modo descrittivo per tracciare chiaramente l'evoluzione delle singole feature e i refactoring architetturali.

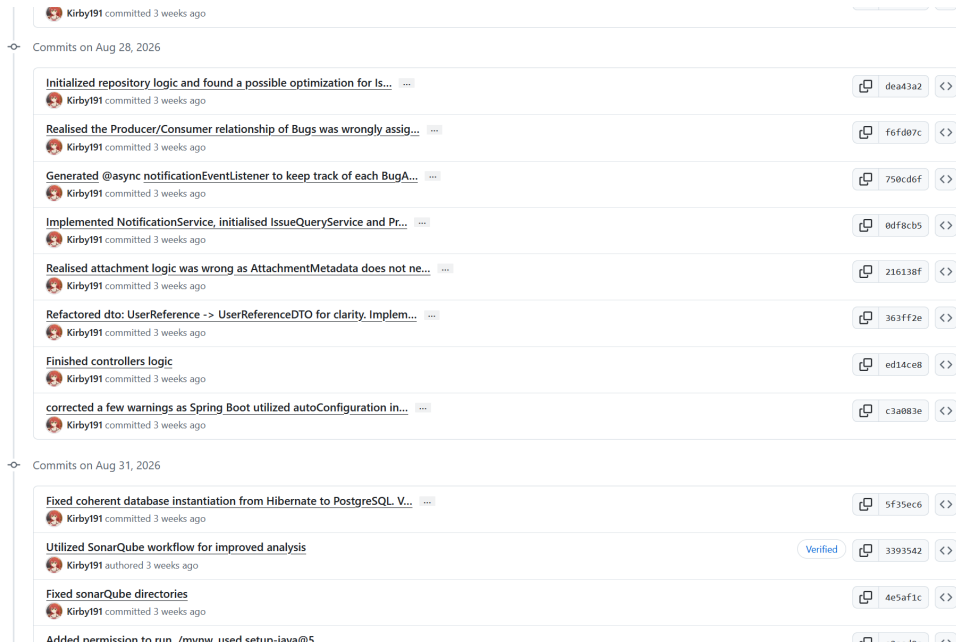


Figura 27: Estratto della cronologia dei commit sul branch main di BugBoard26

7 Code Quality e Analisi Statica

7.1 Strumenti di Analisi

Per garantire e monitorare costantemente la qualità del codice sorgente è stato adottato **SonarQube Cloud**, uno degli strumenti leader nell'industria per l'analisi statica.

7.2 Architettura dell'Analisi CI/CD

Rispetto a una configurazione monolitica tradizionale, l'integrazione di SonarQube tramite GitHub Actions è stata progettata per riflettere l'architettura del sistema.

L'analisi non tratta il back-end come un unico blocco, ma lo separa logicamente e fisicamente nei suoi due domini principali: *auth-service* e *core-service*. Per ottenere questo risultato in modo elegante, la pipeline CI/CD è stata divisa in due workflow distinti, governati tramite il parametro di filtraggio **paths** di GitHub Actions.

Questa scelta architetturale offre due vantaggi fondamentali al sistema:

- **Indipendenza dei Quality Gate:** Ogni microservizio possiede il proprio ciclo di vita e la propria metrica di validazione. Un eventuale abbassamento della code coverage o l'introduzione di un code smell nell'*auth-service* non invalida lo stato del *core-service*, garantendo isolamento e rilasci indipendenti.
- **Ottimizzazione delle risorse CI/CD:** L'analisi di un microservizio viene innescata *se e solo se* il codice sorgente di quello specifico dominio ha subito modifiche. Questo approccio event-driven evita scansioni ridondanti, riducendo drasticamente i tempi di build e il consumo dei runner cloud di GitHub. Questa scelta è stata derivata anche dal fatto che la repository Github non ospita solo il codice sorgente, ma bensì anche diversi artefatti di documentazione.

7.3 Risultati dell'Analisi

I report generati da SonarQube confermano la correttezza delle scelte implementative e l'alta qualità del codice prodotto.

È Importante sottolineare che dati i requisiti di testing imposti dalla traccia del progetto, sarebbe stato impossibile raggiungere una Coverage soddisfacente per far passare il quality Gate di SonarQube (80%). Pertanto, si è deciso di non includere la test coverage nell'analisi tramite JaCoCo.

Il processo di continuous inspection ha inoltre guidato il refactoring e la messa in sicurezza di diverse logiche applicative. Grazie all'analisi statica sono state individuate e risolte problematiche di sicurezza quali potenziali vulnerabilità di *Path Traversal* durante l'archiviazione degli allegati, oltre all'eliminazione di costrutti *Dead Code* e alla standardizzazione dei proxy transazionali all'interno del framework Spring Boot.

I report completi e navigabili sono consultabili pubblicamente ai seguenti indirizzi:

- [Auth Service](#)
- [Core Service](#)

8 Attività di Testing

Al fine di garantire l'affidabilità del sistema e la robustezza della logica di business, è stata progettata una batteria di test di unità mirata alla verifica del Command Layer (**Issue Management Subsystem**) e dell'Audit Trail (**History Subsystem**). I test sono stati implementati tramite **JUnit** e **Mockito**, garantendo l'isolamento dei componenti.

8.1 Input Space Partitioning Functionality-Based

La strategia di progettazione dei test adottata è l'Input Space Partitioning (ISP) con approccio *functionality-based*. L'input space di ciascun metodo viene partizionato in classi di equivalenza derivanti dalle specifiche funzionali e dai vincoli di dominio, analizzando ogni parametro e il contesto di sicurezza (es. utente autenticato). L'ipotesi di base è che tutti i valori appartenenti alla stessa classe producano il medesimo comportamento logico.

8.2 Classi di equivalenza individuate

8.2.1 Metodo: assignBug(Long id, AssignBug request)

Parametro	Classe di equivalenza	Valore Rappr.	Test Case
id	Issue esistente di tipo BUG	1L	TC-01, TC-03, TC-04
id	Issue esistente non-BUG (Feature)	2L (Feature)	TC-02
id	Issue inesistente nel DB	999L	TC-05
request	Contiene un assigneeId valido	200L	TC-01, TC-02, TC-04
request	Contiene un assigneeId inesistente	888L	TC-03
(Security)	L'utente autenticato è ROLE_ADMIN	adminId	TC-01, TC-02, TC-03, TC-05
(Security)	L'utente autenticato è ROLE_UTENTE	userId	TC-04

8.2.2 Metodo: `updateIssue(Long id, UpdateIssue request, MultipartFile file)`

Nota architetturale: Rispetto al design iniziale, la firma di questo metodo è stata estesa introducendo il terzo parametro opzionale `MultipartFile file`. Originariamente, il metodo accettava solo l'ID e il DTO della request testuale (`UpdateIssue`). L'aggiunta del `MultipartFile` permette di supportare l'upload e la sovrascrittura dell'allegato (immagine) anche in fase di modifica, estendendo la Funzionalità 2 senza alterare il payload JSON principale e rispettando i principi RESTful. Il Multipart file rappresentato è di tipo `Mock`

Parametro	Classe di equivalenza	Valore Rappr.	Test Case
<code>id</code>	Issue esistente nel DB	1L	TC-06, TC-07, TC-08, TC-09b
<code>id</code>	Issue inesistente nel DB	999L	TC-09
<code>request</code>	Stato modificato (es. da TODO a IN_PROGRESS)	IN_PROGRESS	TC-06, TC-08, TC-09
<code>request</code>	Stato inalterato o null (solo update testi)	null	TC-07, TC-09b
<code>file</code>	File presente (non vuoto)	MultipartFile*	TC-09b
<code>file</code>	File assente (null)	null	TC-06, TC-07, TC-08, TC-09
<i>(Security)</i>	Autore della issue (Reporter) o Admin	reporterId	TC-06, TC-07, TC-09, TC-09b
<i>(Security)</i>	Utente estraneo non autorizzato	otherUserId	TC-08

8.2.3 Metodo: `recordEvent(Long bugId, Long authorId, AuditAction action, String details)`

Parametro	Classe di equivalenza	Valore Rappr.	Test Case
<code>bugId</code>	ID numerico valido associato al bug	1L	TC-10
<code>authorId</code>	ID dell'utente censito nel sistema	100L	TC-10
<code>action</code>	Enumerazione standard di History	CREATED	TC-10
<code>details</code>	Stringa descrittiva valida	"Dettaglio"	TC-10

8.3 Criterio di copertura: WECT (Each Choice Coverage)

Il criterio di copertura adottato per l'integrazione delle classi di equivalenza è l'**Each Choice Coverage** (o *Weak Normal Equivalence Class Testing* - WECT). Questo criterio stabilisce che ogni classe di equivalenza di ciascun parametro debba essere coperta da almeno un caso di test, senza l'obbligo di testare tutte le combinazioni possibili incrociate.

La scelta del WECT è motivata dal bilanciamento ottimale tra affidabilità del collaudo e costo computazionale: previene l'esplosione esponenziale del numero di test (*combinatorial explosion*) pur garantendo che ogni partizione del dominio, compresi gli scenari di *Error Path* (`InvalidIssueDomainException`, `UnauthorizedActionException`, ecc.), venga esercitata. Sono stati implementati **11 Test Case** totali (incluso il collaudo del caricamento file in formato multipart) che soddisfano pienamente tale copertura.

8.4 Strategie non adottate e motivazioni

Strategia	Descrizione	Motivazione dell'esclusione
Boundary Value Analysis	Test sui confini delle classi numeriche o di range (min, max).	I parametri di input sono identificatori di database (<code>Long</code>), istanze DTO o enumerazioni. Mancano valori continui in cui i confini (es. $x > 0$) causino variazioni logiche significative.
All Combinations Coverage (SECT)	Copertura totale del prodotto cartesiano delle classi di equivalenza.	Crescita esponenziale dei test (es. decine di casi per <code>assignBug</code>) senza un aumento giustificabile della probabilità di individuare nuovi difetti (<i>faults</i>).
Robustness Testing (R-WECT)	Test con valori esterni al dominio logico (null, stringhe vuote).	La validazione dei formati primari (es. parametri nulli non ammessi) è delegata a monte a <i>Bean Validation</i> (<code>@Valid</code>) nel Controller Layer; il Service Layer riceve dati già strutturati.

8.5 Metriche di Copertura Strutturale (White-Box)

Al fine di validare internamente l'implementazione dei tre metodi e garantire che non esistano "linee di codice morte", l'analisi *Black-Box* è stata affiancata dal **White-Box Testing** per il calcolo della copertura strutturale.

Metodo	Statement Coverage	Branch/ Decision Cov.	Dettaglio Copertura Logica
assignBug	100%	100%	Copertura totale del controllo di dominio <code>bug.getType() != BUG</code> , coprendo sia il ramo positivo (persistenza ed emissione evento) sia l'interruzione anticipata con eccezione.
updateIssue	100%	100%	Copertura esaustiva del costrutto <code>if(isStatusChanged)</code> e del nuovo ramo logico di gestione allegati <code>if(file != null)</code> collaudato dal caso TC-09b . È testato sia il blocco che genera l'audit <code>STATUS_CHANGED</code> , sia il ramo <code>else</code> per l'audit <code>UPDATED</code> .
recordEvent	100%	N/A	Metodo puramente sequenziale senza diramazioni condizionali. Coperto interamente dall'invocazione di TC-10 e validazione dell'uso del Proxy <code>getReferenceById</code> .